Unit-4
Transport and Session Layer Protocols

- **Transport Layer**
  - **TCP**
  - **MPTCP**
  - **UDP**
  - **DCCP**
  - **SCTP**
  - **TLS**
  - **DTLS**

- **Session Layer**
  - **HTTP**
  - **CoAP**
  - **XMPP**
  - **AMQP**
  - **MQTT**

TCP, UDP, MPTCP, DCCP, SCTP

01 Why the Transport Layer Matters in IoT

02 TCP — reliable, connection-oriented delivery

03 UDP — lightweight, connectionless delivery

04 SCTP — multi-streaming and multi-homing

05 DCCP — congestion-controlled datagrams

06 MPTCP — TCP across multiple paths

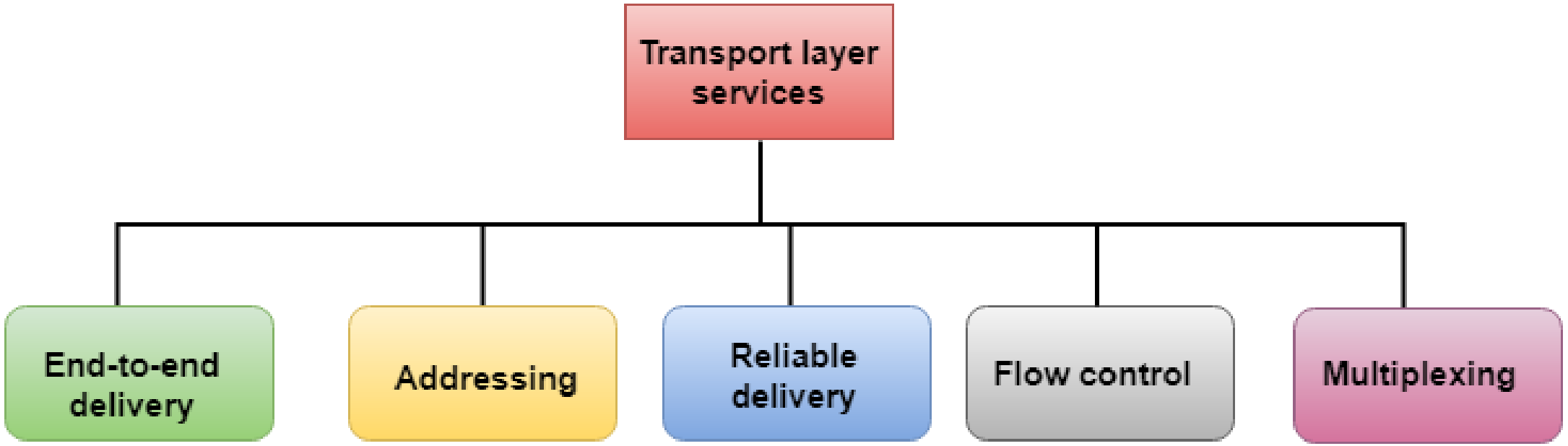
07 Side-by-side comparison and protocol selection

FOUNDATIONS

Role of the Transport Layer in IoT

- **P** **Process-to-Process Delivery** — Ports multiplex many sensor/actuator apps over one network address.
- **R** **Reliability Trade-off** — Guaranteed delivery costs energy, bandwidth and latency on constrained nodes.
- **C** **Congestion & Flow Control** — Keeps low-power, lossy networks (LLNs) from collapsing under traffic.
- **F** **Protocol Fit Shapes the System** — Choice of TCP/UDP/SCTP/DCCP/MPTCP defines an IoT deployment's performance.

- **Transport Layer**



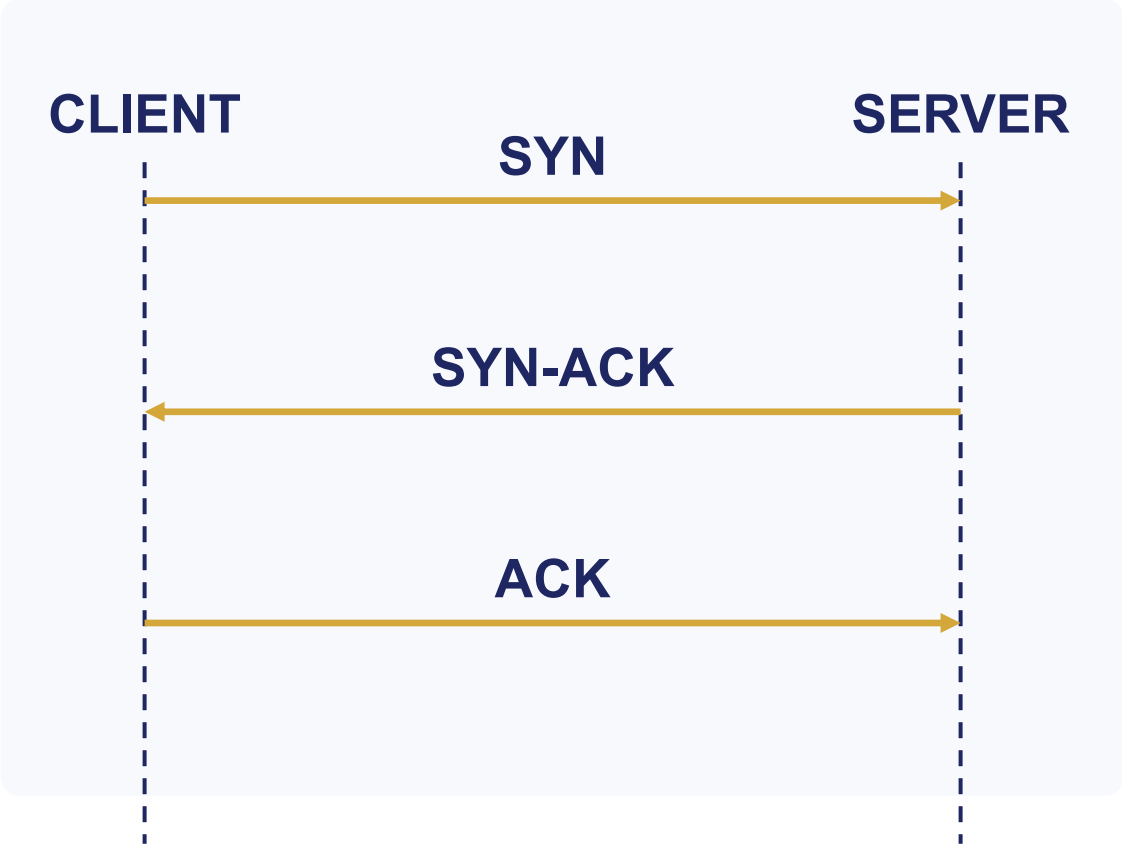
PROTOCOL 01

TCP — Transmission Control Protocol

Key Characteristics

1. Connection-oriented (3-way handshake)
2. Reliable, ordered byte-stream delivery
3. Built-in flow & congestion control
4. 20-byte minimum header — heavier for IoT

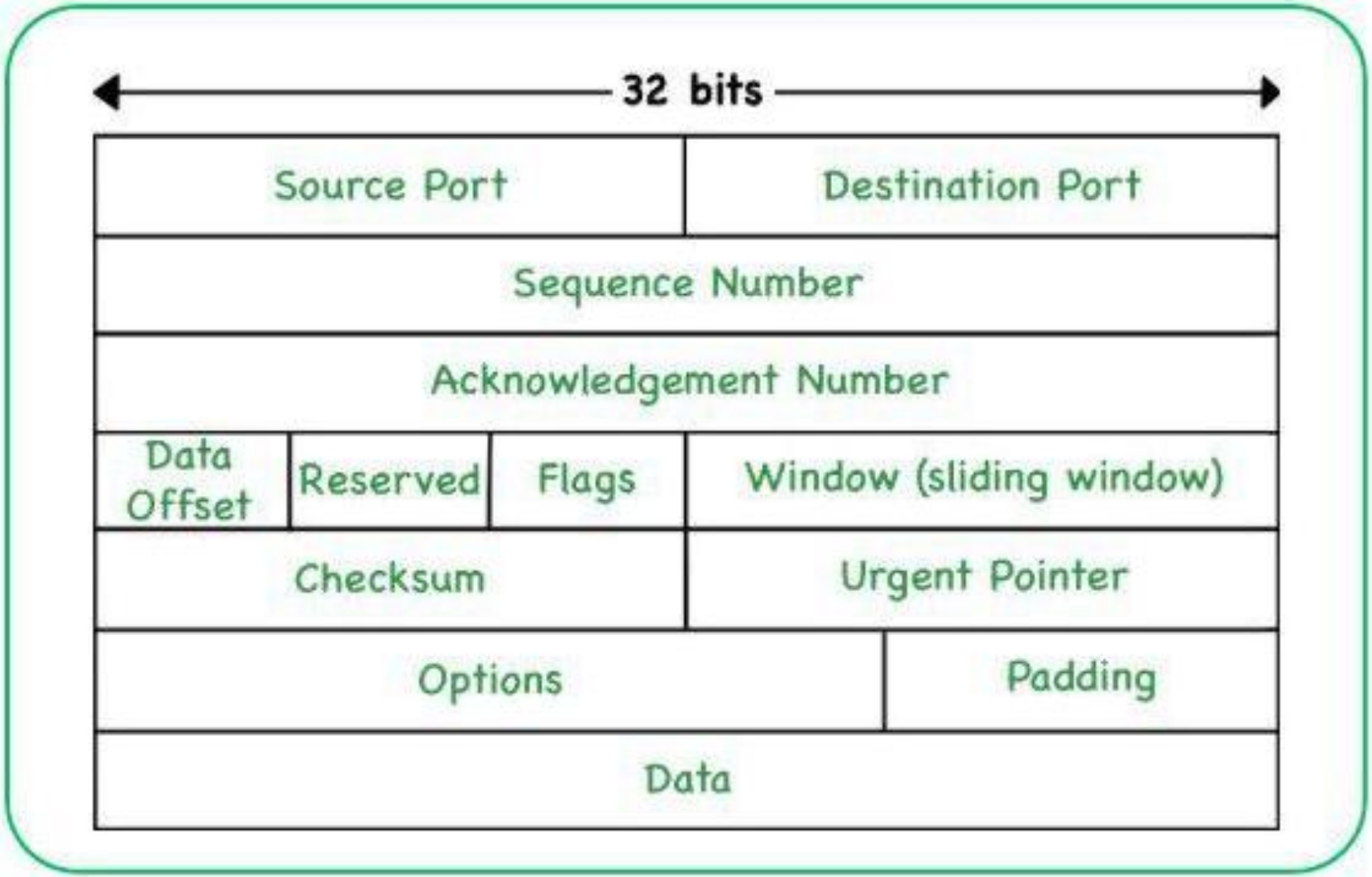
Three-Way Handshake



TCP



TCP-Header format



PROTOCOL 01 · APPLIED

TCP in IoT: Strengths vs. Challenges

Strengths

+ Guaranteed, in-order delivery
+ Proven congestion control
+ Works with firewalls / NAT easily
+ Ideal for firmware & config updates

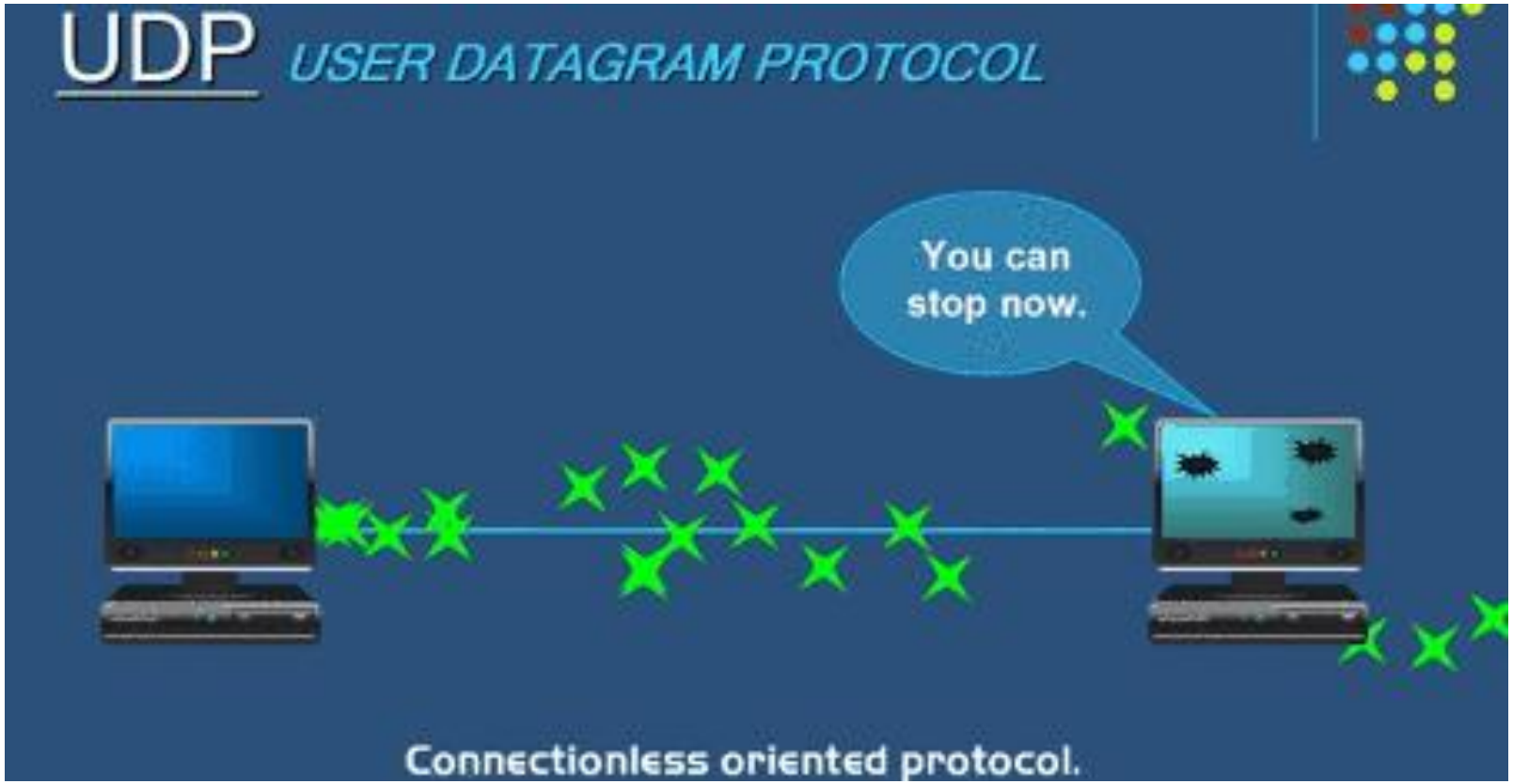
Challenges

– High header & memory overhead
– Handshake adds connection latency
– Retransmissions drain battery power
– Poor fit for lossy radio links

Applications of TCP

- Web Browsing
- File Transfers
- Email
- Remote Access
- Database Access

UDP


UDP
USER DATAGRAM PROTOCOL
You can stop now.
Connectionless oriented protocol.

PROTOCOL 02

UDP — User Datagram Protocol

Key Characteristics

1. Connectionless — no handshake required
2. Minimal 8-byte header
3. No delivery, ordering, or duplication guarantee
4. Supports broadcast & multicast

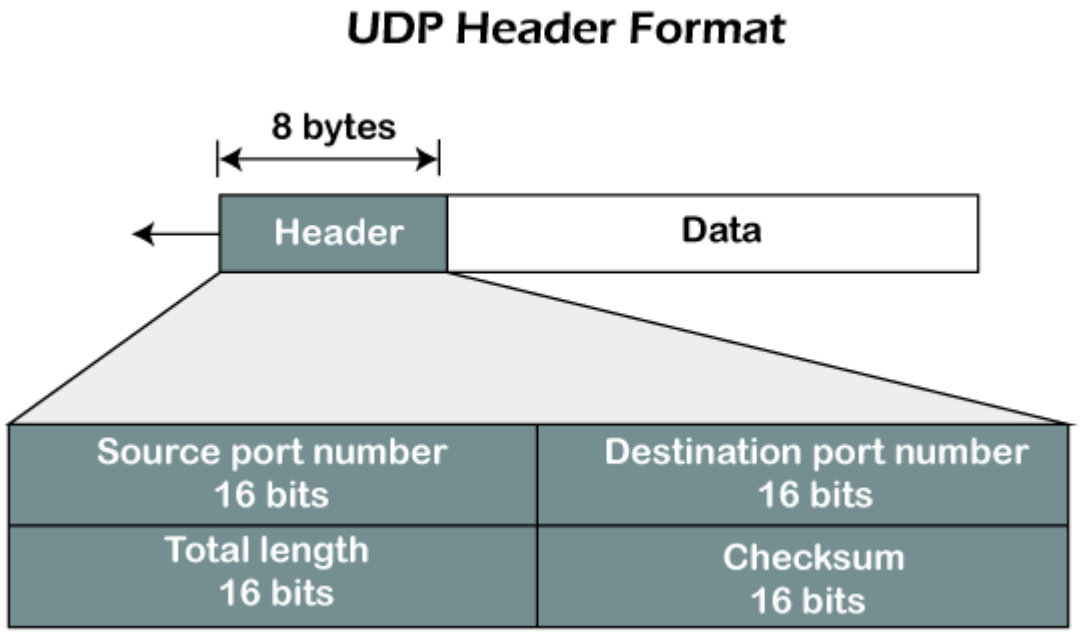
8 B
Header size

0
Handshake round trips

Best
Effort delivery model

Low
Energy & latency cost

UDP-Header format



PROTOCOL 02 · APPLIED

UDP in IoT: Where It Is Used

C **CoAP Messaging** — The Constrained Application Protocol runs over UDP for lightweight REST calls.

D **Discovery & Multicast** — mDNS / service discovery relies on UDP broadcast across sensor networks.

S **Streaming Telemetry** — Loss-tolerant sensor readings favor speed over guaranteed delivery.

L **Low-Power Radios** — 6LoWPAN / LLNs benefit from UDP's tiny header and no connection state.

Applications of UDP:

- Used for some routing update protocols like RIP(Routing Information Protocol).
- Normally used for real-time applications
- VoIP (Voice over Internet Protocol) services, such as Skype and WhatsApp
- **DNS** also uses UDP for its query/response messages.
- **DHCP (Dynamic Host Configuration Protocol)** uses UDP to dynamically assign IP addresses to devices on a network.

CHECKPOINT

TCP vs. UDP at a Glance

Feature	TCP	UDP
Connection	Yes (handshake)	No
Reliability	Guaranteed	Best effort
Ordering	In order	Not guaranteed
Header size	20 bytes+	8 bytes
Overhead	Higher	Very low
Typical IoT use	Firmware updates	CoAP, telemetry

PROTOCOL 03

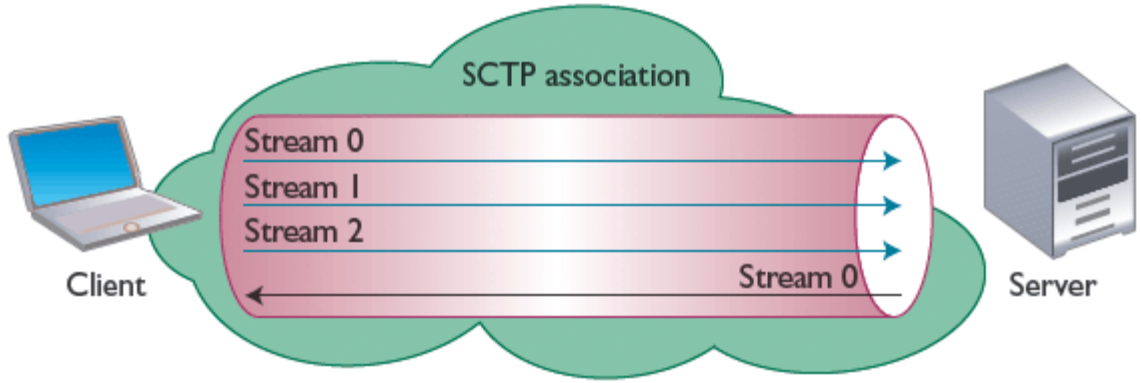
SCTP — Stream Control Transmission Protocol

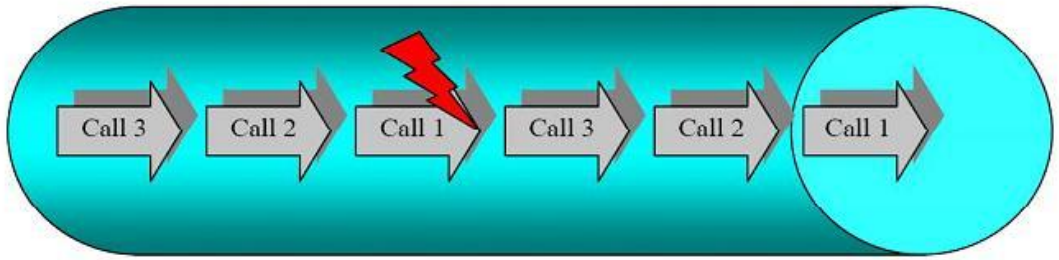
Key Characteristics

1. Message-oriented — preserves record boundaries
2. Multi-streaming avoids head-of-line blocking
3. Multi-homing — multiple IP paths per session
4. 4-way handshake resists SYN-flood attacks

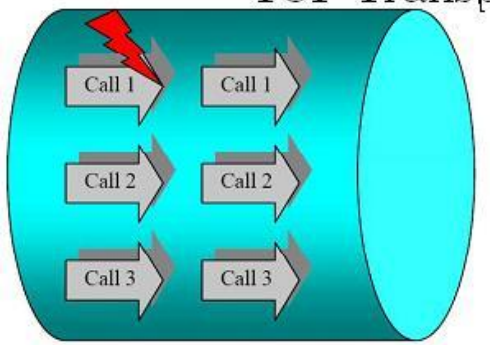
SCTP

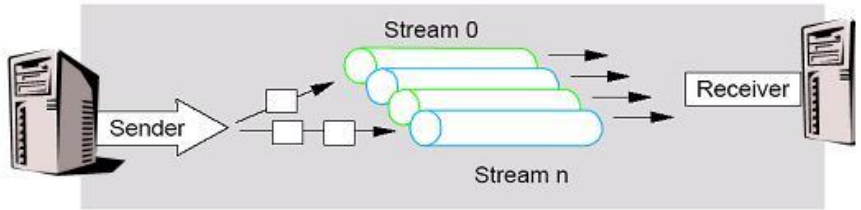
SCTP-Multi Streamed





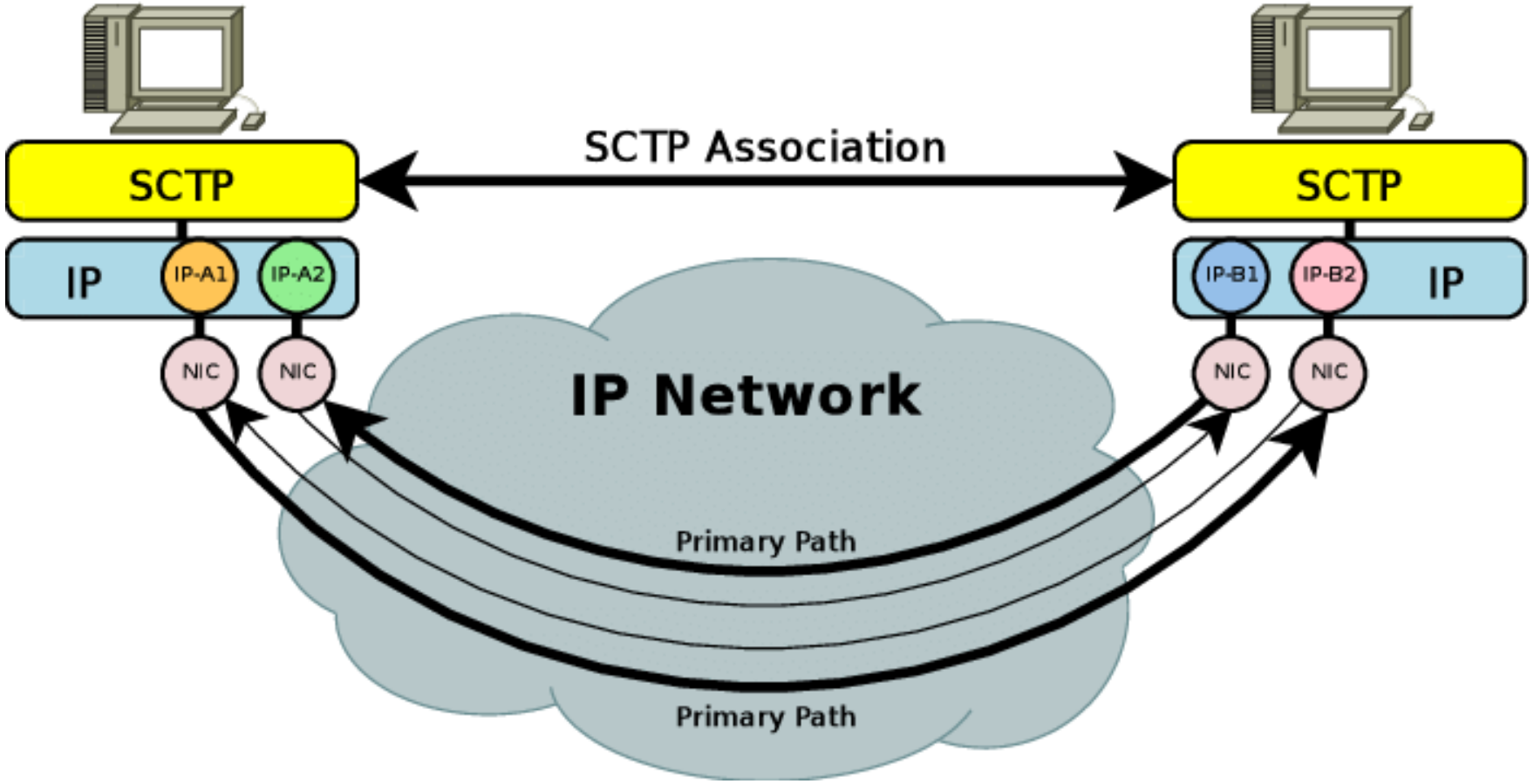
TCP Transport - Byte Oriented





SCTP

Multi homing with SCTP



PROTOCOL 03 · APPLIED

SCTP in IoT: Use Cases

G **Resilient Gateways** — Multi-homing keeps IoT gateway links alive when one network path fails.

S **Parallel Sensor Streams** — Multiple independent streams avoid blocking one sensor's data on another's loss.

T **Signaling & Control** — Telecom-grade signaling for critical infrastructure and industrial IoT control planes.

PROTOCOL 04

DCCP — Datagram Congestion Control Protocol

Key Characteristics

1. Unreliable datagrams — like UDP
2. Connection setup/teardown — like TCP
3. Built-in, pluggable congestion control
4. No retransmission — timely over complete

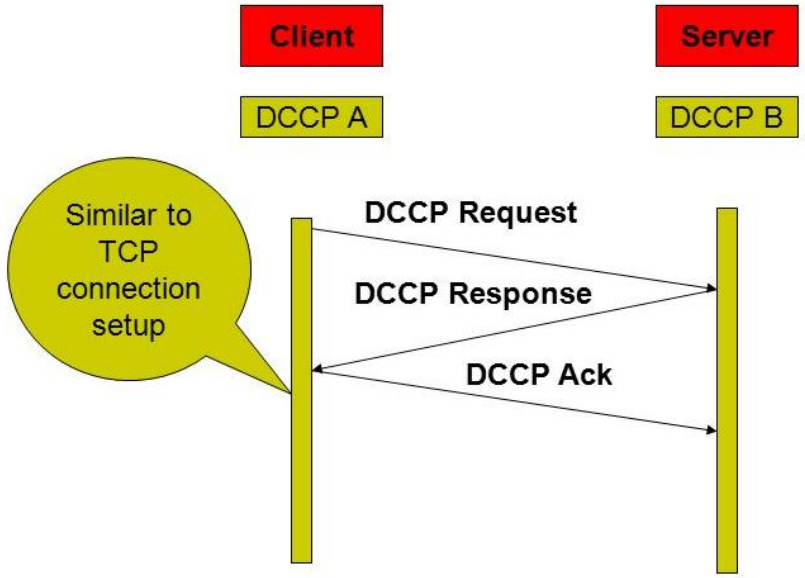
Congestion Control Profiles

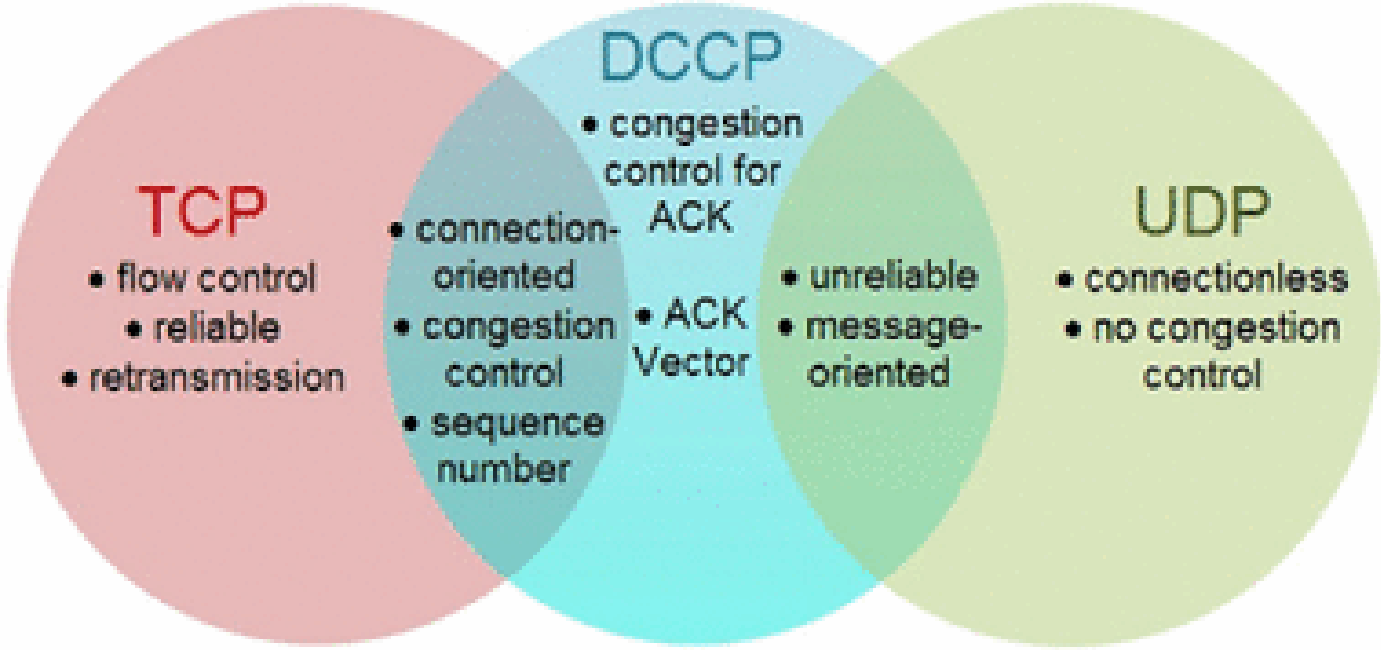
CCID 2 — TCP-like control

CCID 3 — TFRC, smooth rate for streaming

DCCP

DCCP Connection Setup





PROTOCOL 04 · APPLIED

DCCP in IoT: Use Cases

V **IoT Video & Audio** — Camera / voice feeds need timely frames, not perfect retransmitted ones.

N **Network-Friendly Telemetry** — Congestion awareness stops sensor floods from overwhelming shared LLNs.

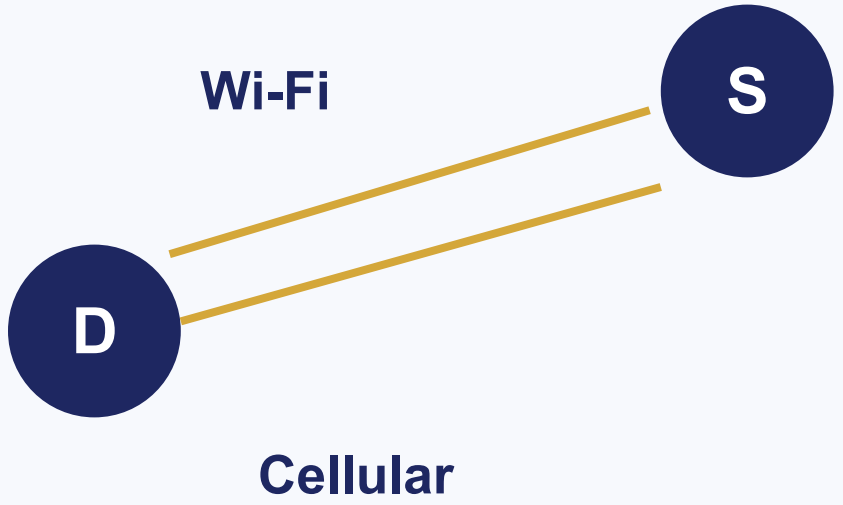
E **Edge Analytics Streams** — Continuous metric streams tolerate loss but must react to congestion fast.

PROTOCOL 05

MPTCP — Multipath TCP

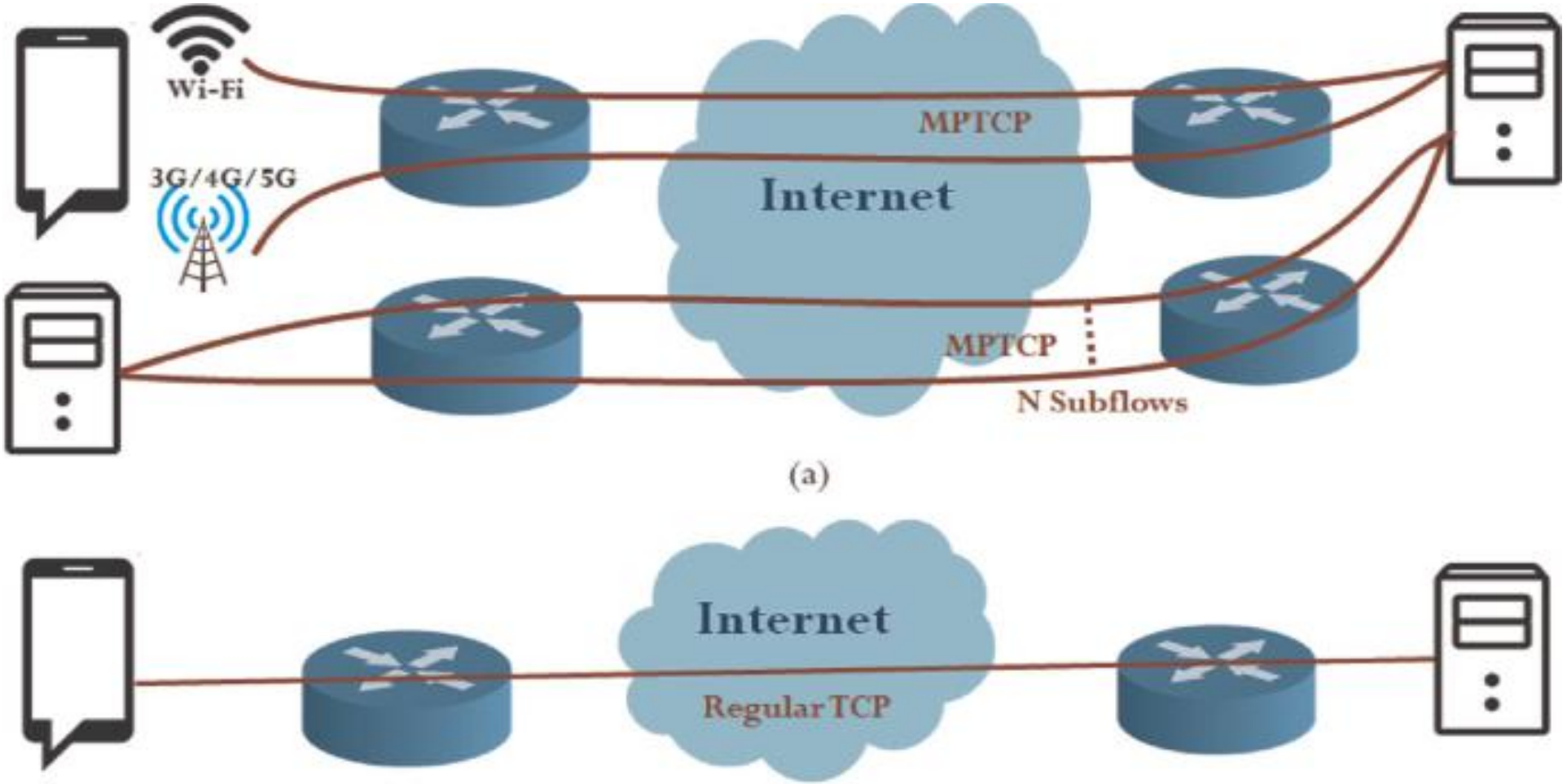
Key Characteristics

1. Extends TCP to use several paths at once
2. Aggregates throughput across interfaces
3. Seamless failover — no dropped connection
4. Backward compatible with plain TCP



Device with two interfaces ↔ single server
(one logical MPTCP connection)

MPTCP


Wi-Fi
3G/4G/5G
MPTCP
Internet
MPTCP
N Subflows
(a)
Internet
Regular TCP

MPTCP

It achieves this by using TCP options to achieve the following goals:

- Establish a new MPTCP connection.
- Add subflows to an MPTCP connection.
- Transmit data on the MPTCP connection.

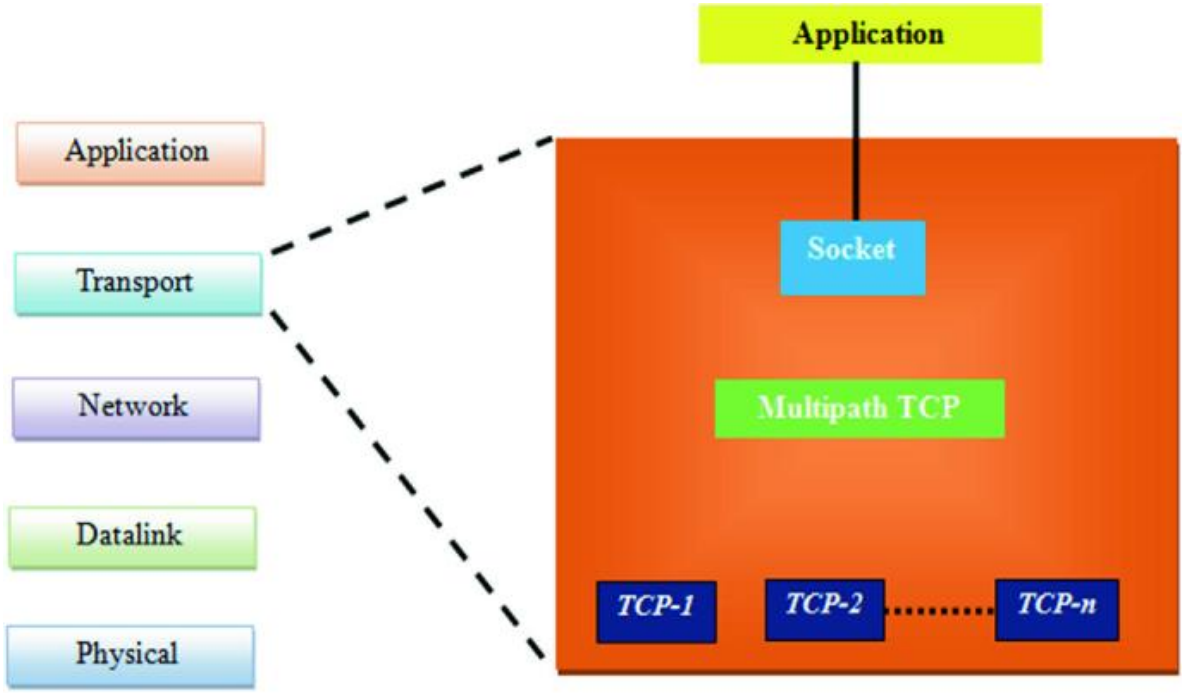
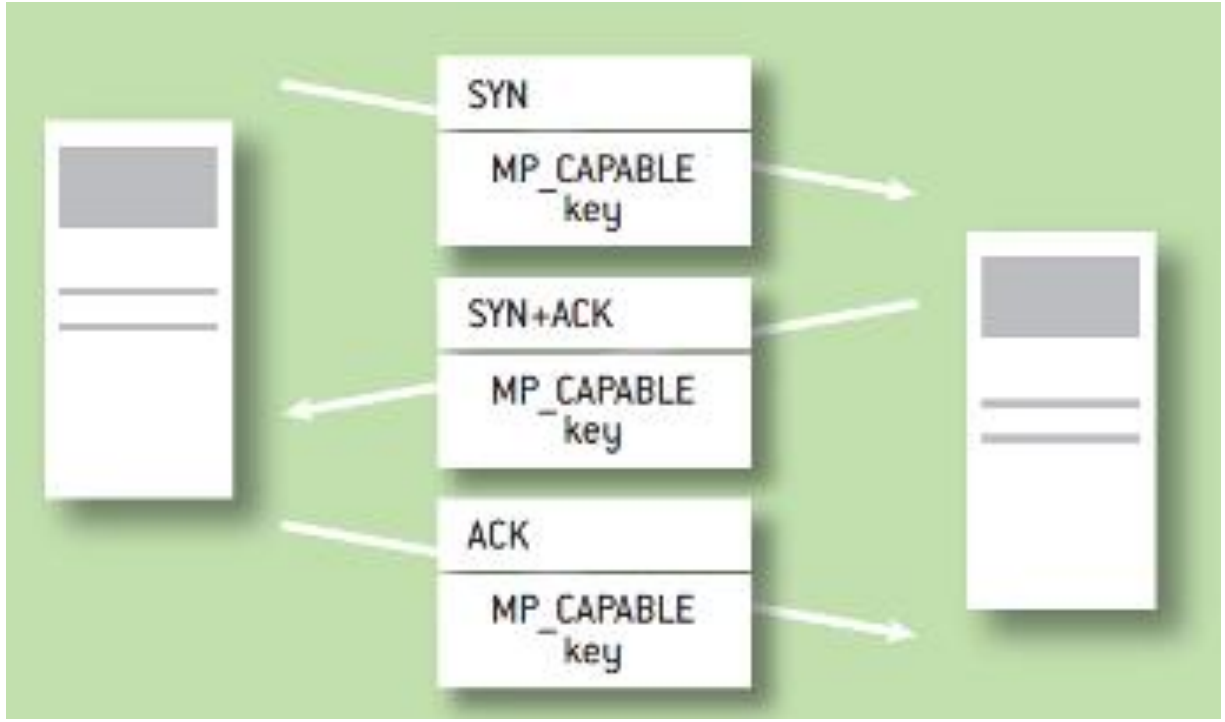


figure 1:Architecture of MPTCP

MPTCP

Connection Establishment

MPTCP needs to be able to link each subflow to an existing MPTCP connection.
For this, MPTCP assigns a locally unique token to each connection.
When a new subflow is added to an existing MPTCP connection, the MP_JOIN option of the SYN segment contains the token of the associated MPTCP connection shown in figure 3.



three-way handshake-MPTCP

PROTOCOL 05 · APPLIED

MPTCP in IoT: Use Cases

G **Multi-Interface Gateways** — Edge gateways with Wi-Fi + cellular stay online if one link degrades.

I **Industrial & Healthcare IoT** — Mission-critical control loops need connection resilience, not just speed.

T **Higher Aggregate Throughput** — Bulk data (video, logs) transfers faster by combining path capacity.

CHECKPOINT

Five Protocols, Side by Side

	TCP	UDP	SCTP	DCCP	MPTCP
Connection	Yes	No	Yes	Yes	Yes
Reliable	Yes	No	Yes	No	Yes
Ordered	Yes	No	Per stream	No	Yes
Multi-path	No	No	Multi-home	No	Yes
Best for	Bulk data	CoAP	Signaling	Media	Resilience

DECISION GUIDE

Choosing the Right Transport Protocol

R Need guaranteed, ordered delivery? **Choose TCP.**

F Need speed, tolerate some loss? **Choose UDP.**

M Need multiple independent streams or paths? **Choose SCTP.**

C Need timely media with congestion control? **Choose DCCP.**

P Need TCP reliability across multiple links? **Choose MPTCP.**

TLS

TLS in the Transport Layer

Transport Layer Security (TLS) is a cryptographic protocol designed to provide secure communication over a network.

While it operates **on top of the transport layer**, its role is closely tied to the transport layer protocols like **TCP**, ensuring confidentiality, integrity, and authenticity for transmitted data.

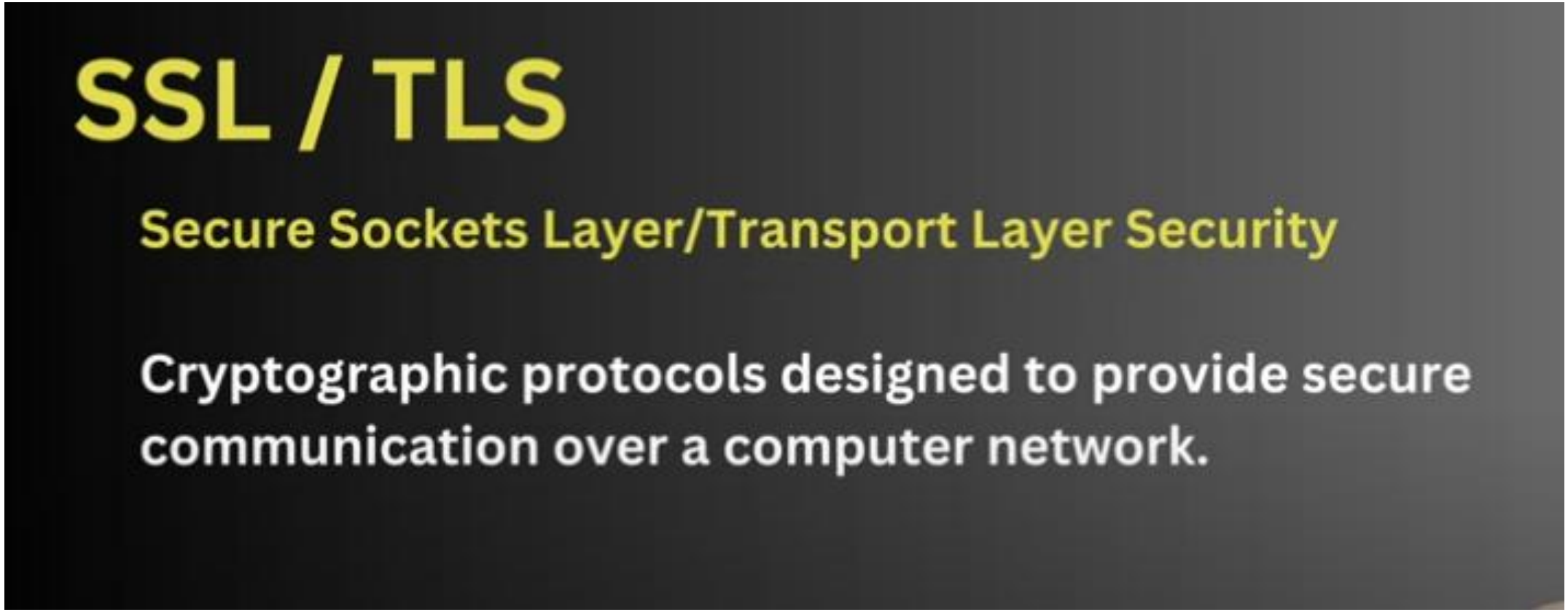


TLS

Why Security is Required For Data?

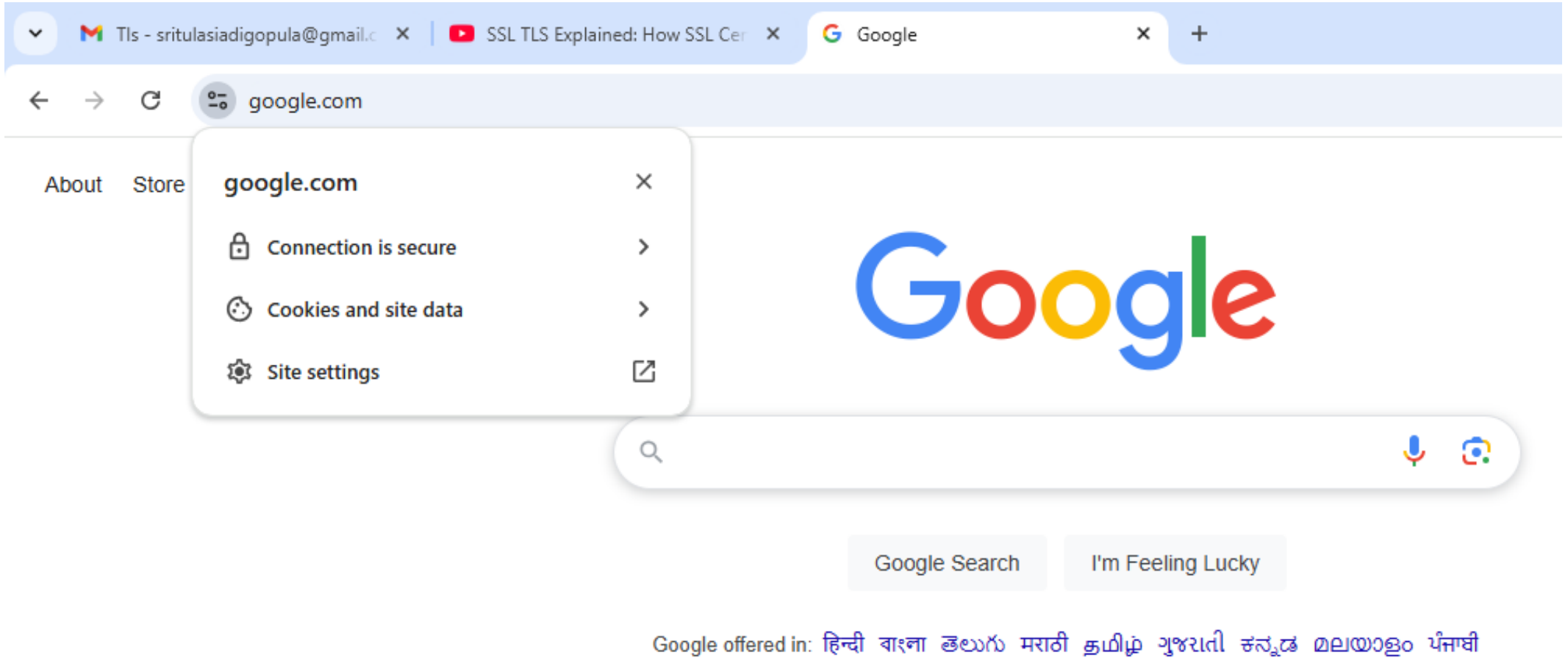


TLS

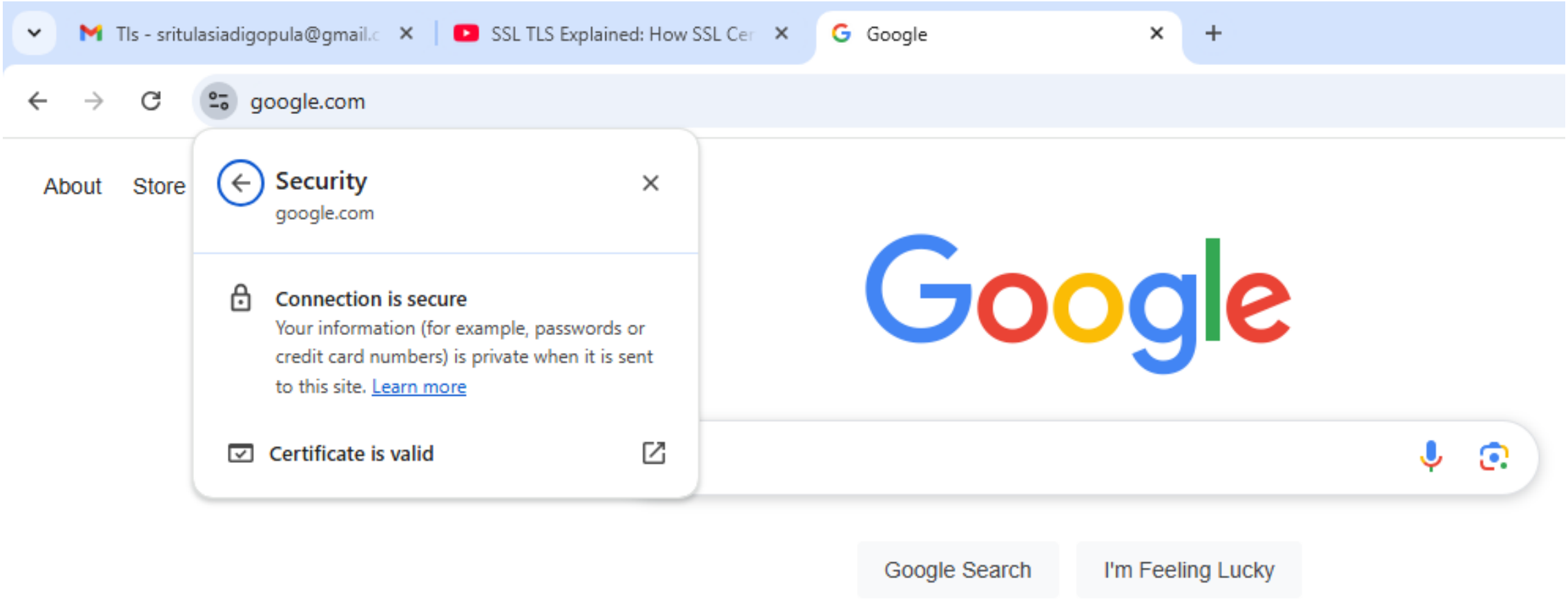


TLS is the successor to SSL

TLS


Tls - sritulasiadigopula@gmail.c
SSL TLS Explained: How SSL Cer
Google
google.com
About
Store
google.com
Connection is secure
Cookies and site data
Site settings
Google
Google Search
I'm Feeling Lucky
Google offered in: हिन्दी বাংলা తెలుగు मराठी தமிழ் ગુજરાતી ಕನ್ನಡ മലയാളം ਪੰਜਾਬੀ

TLS

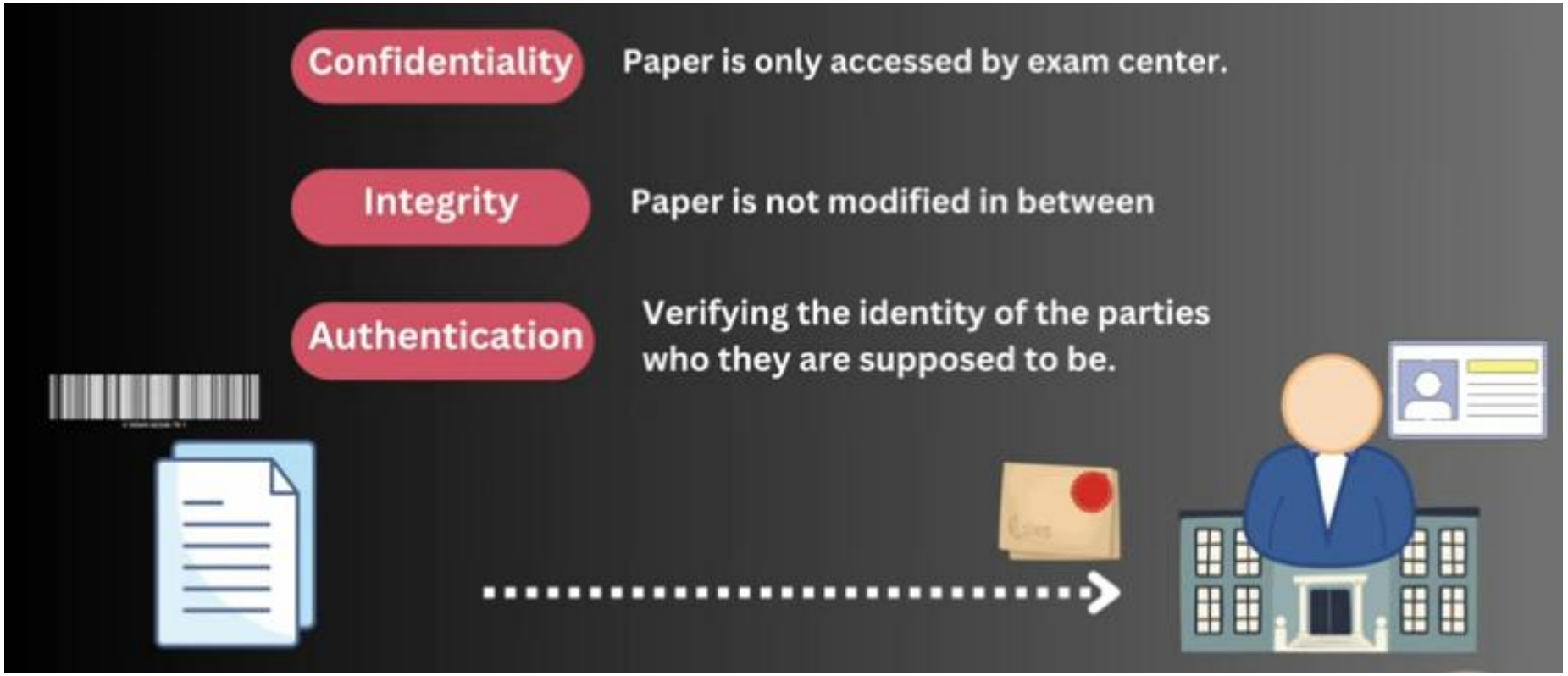

Tls - sritulasiadigopula@gmail.c
SSL TLS Explained: How SSL Cer
Google
google.com
About
Store
Security
google.com
Connection is secure
Your information (for example, passwords or credit card numbers) is private when it is sent to this site. Learn more
Certificate is valid
Google
Google Search
I'm Feeling Lucky
Google offered in: हिन्दी বাংলা తెలుగు मराठी தமிழ் ગુજરાતી ಕನ್ನಡ മലയാളം ਪੰਜਾਬੀ

TLS


How SSL/TLS Secure data?
Encrypt data and ensure its
• integrity,
• confidentiality, and
• authenticity
between a client and server.

TLS

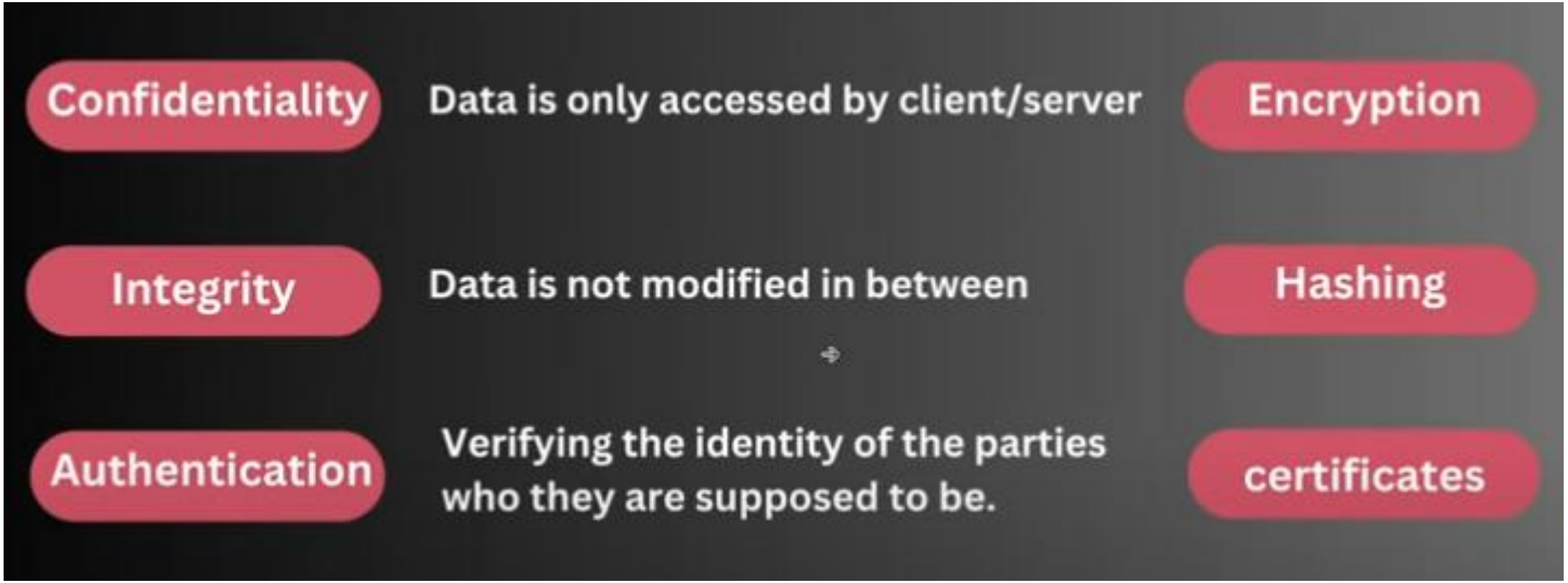
Security-Question Paper Example



Question Paper **Exam Center**

TLS

How can we achieve Security?



TLS


Encryption
Converting plaintext information into a coded form (cipher text).

Example:

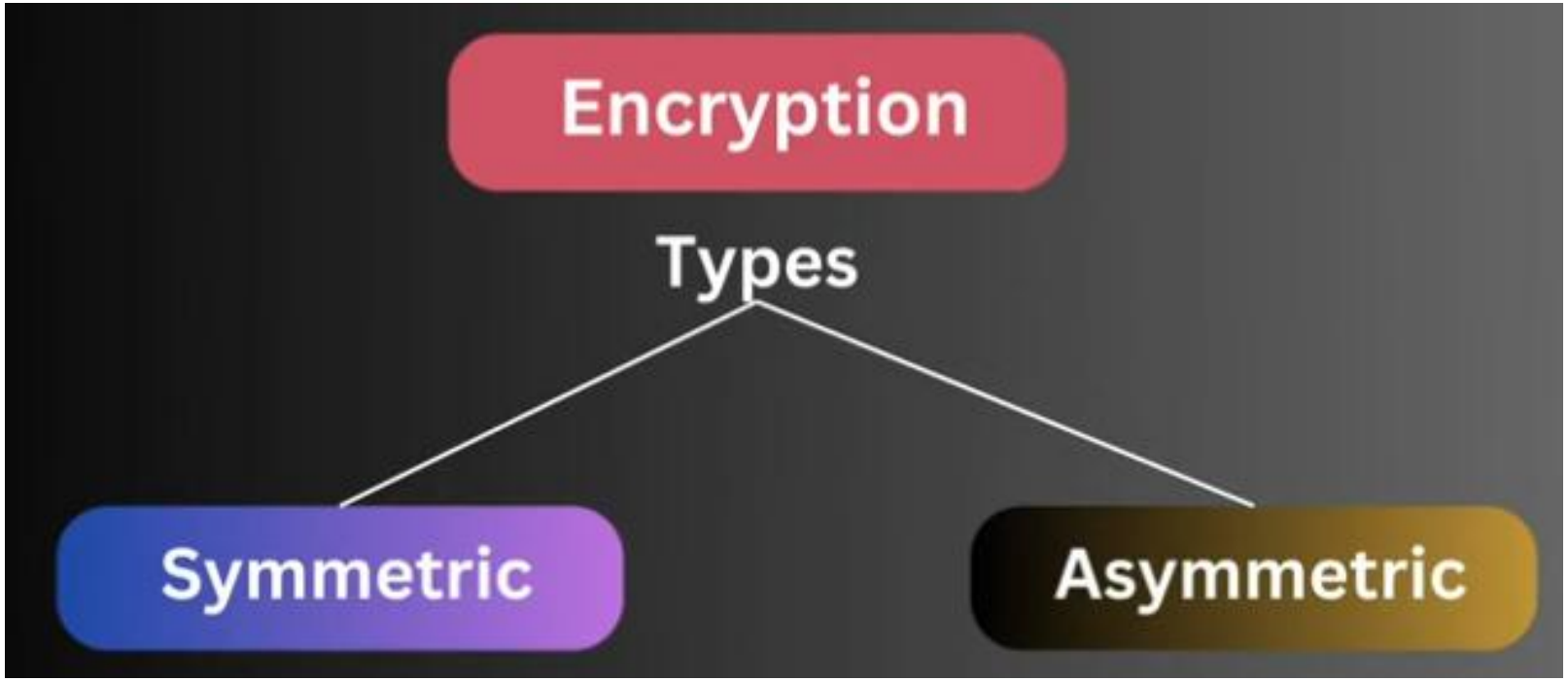

ABCDEFGHIJKLMNOPQRSTUVWXYZ
Encrypt
DEMO --> GHPR
(Ciphertext, each character shifted by 3 char)

TLS

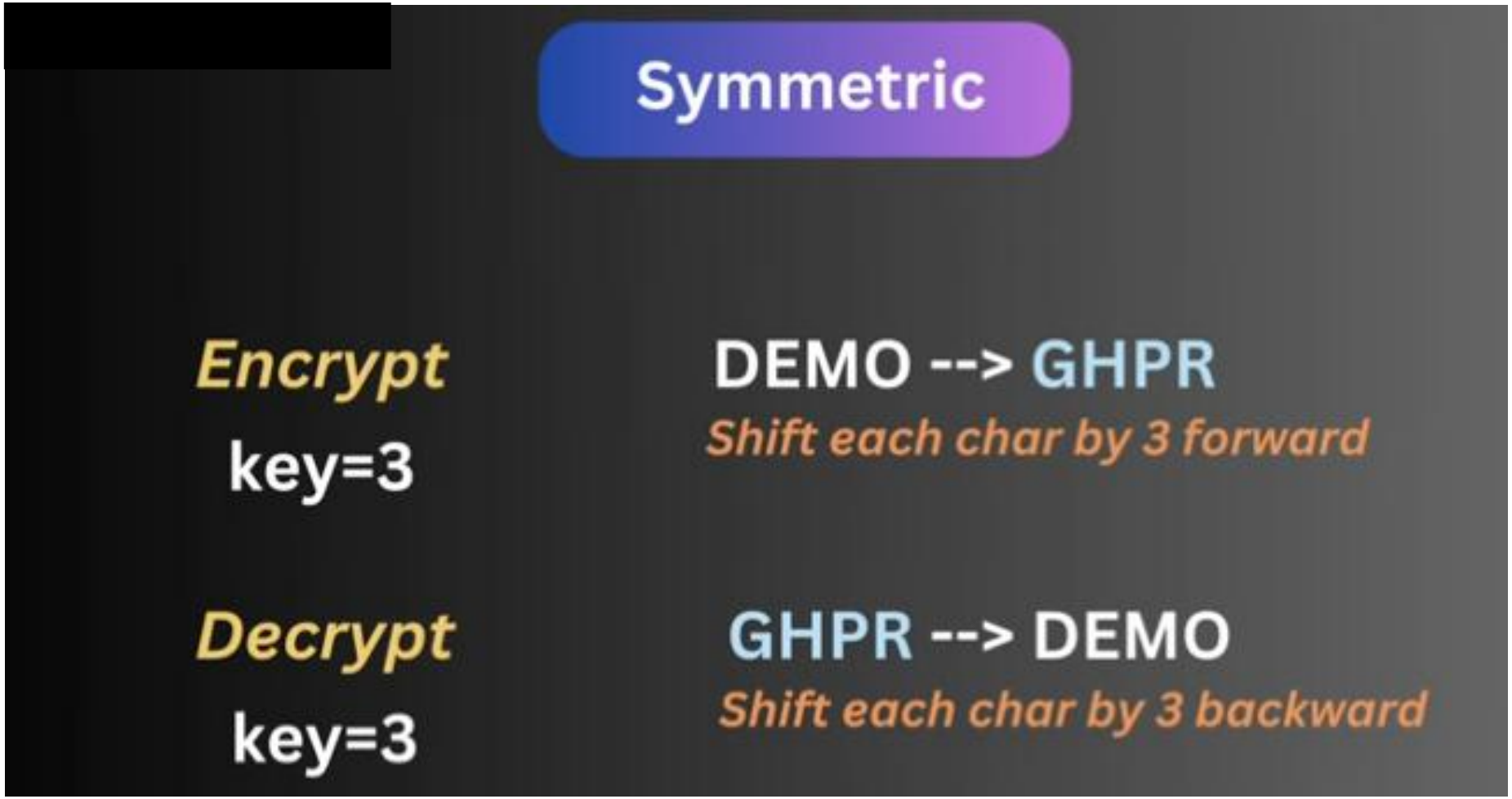
Decryption



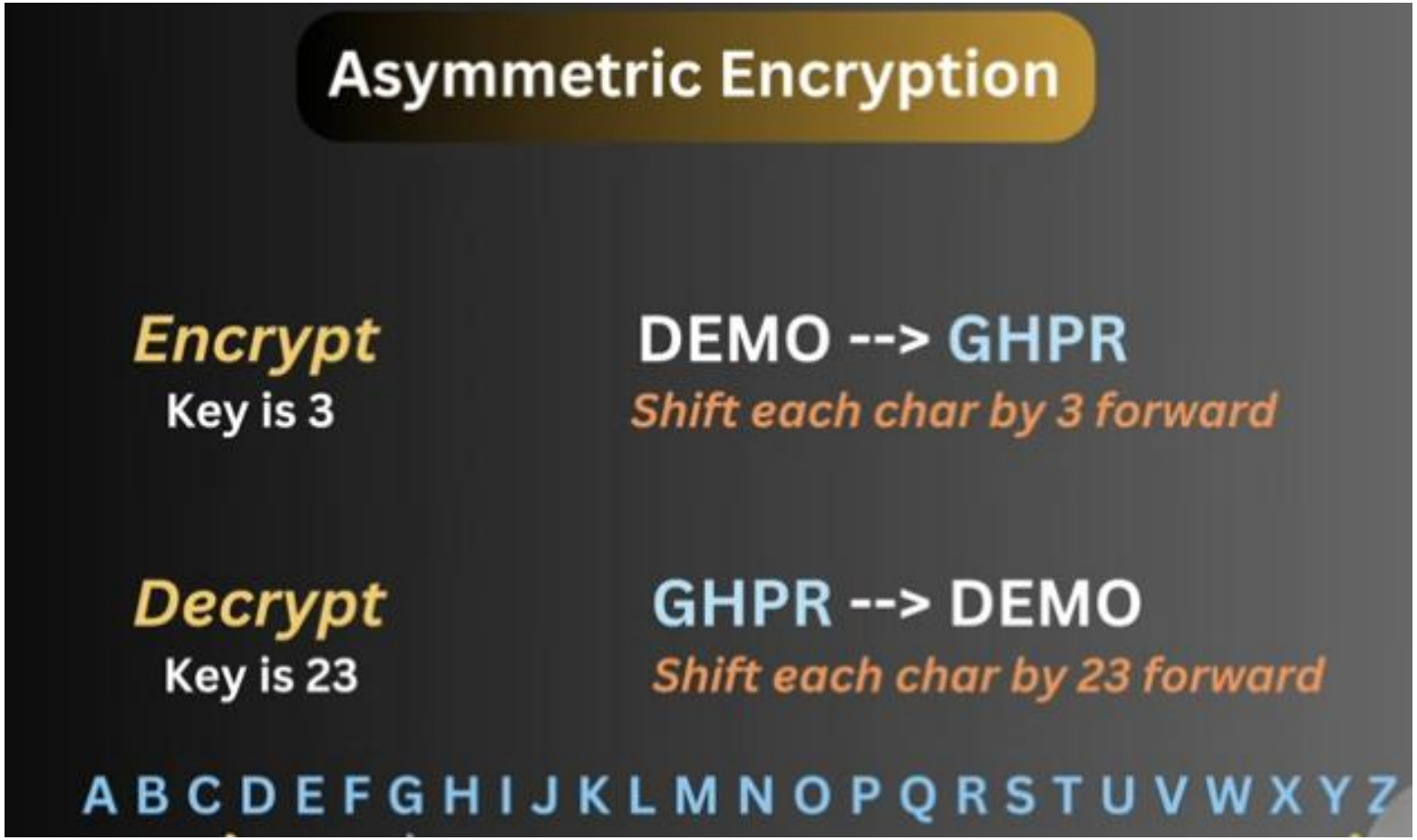
TLS



TLS


Symmetric
Encrypt
key=3
DEMO --> GHPR
Shift each char by 3 forward
Decrypt
key=3
GHPR --> DEMO
Shift each char by 3 backward

TLS


Asymmetric Encryption
Encrypt
Key is 3
DEMO --> GHPR
Shift each char by 3 forward
Decrypt
Key is 23
GHPR --> DEMO
Shift each char by 23 forward
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

TLS

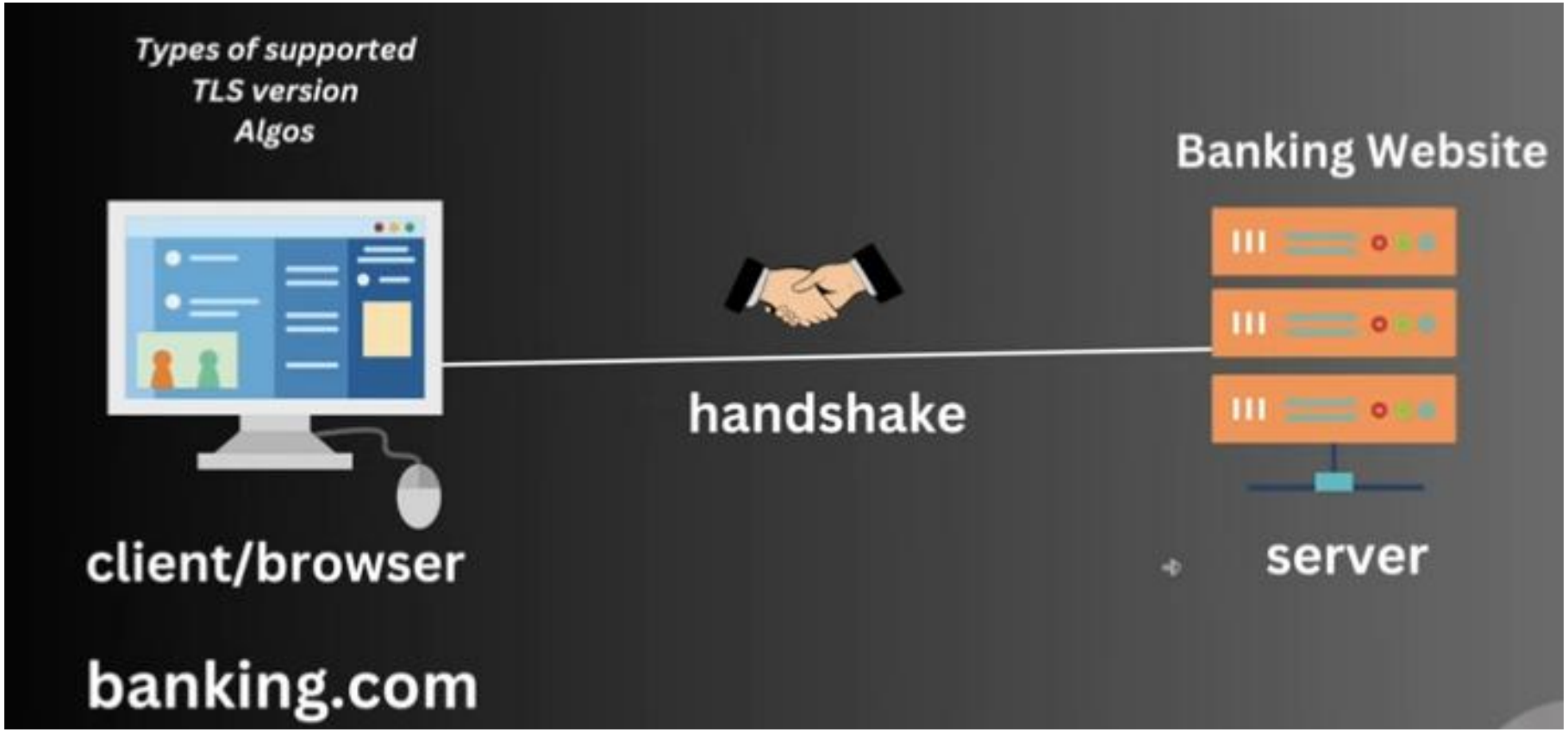
Asymmetric Encryption: Different keys are used for Encryption and Decryption



TLS

Connection Establishment

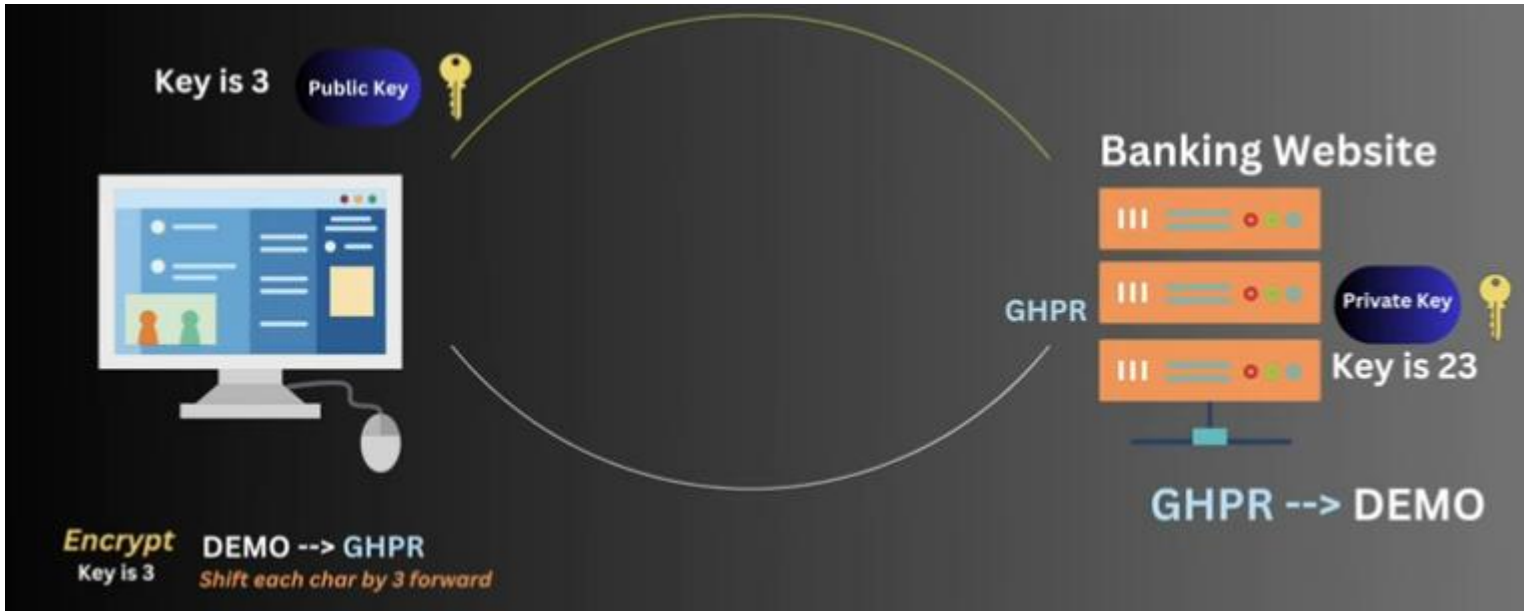
1. Client Hello



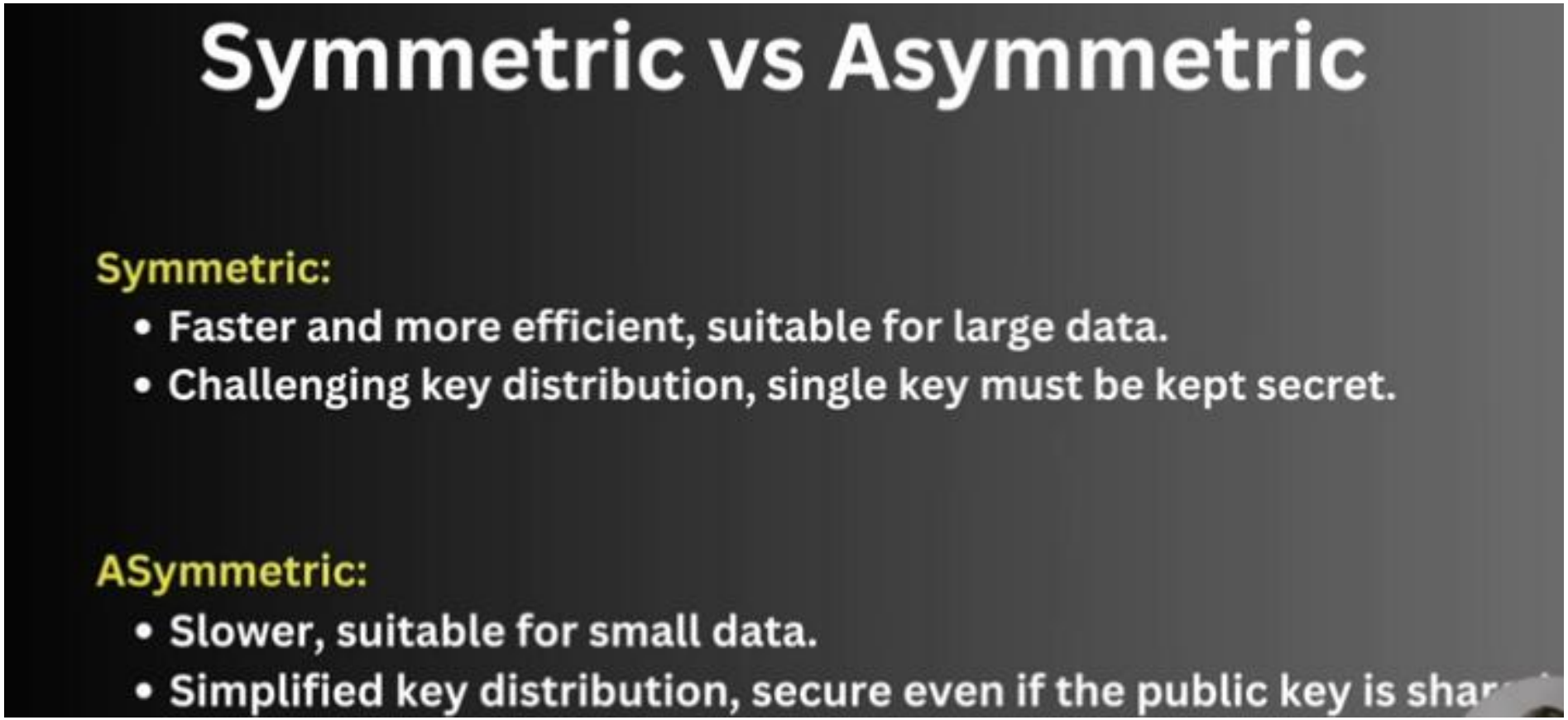
TLS

Connection Establishment

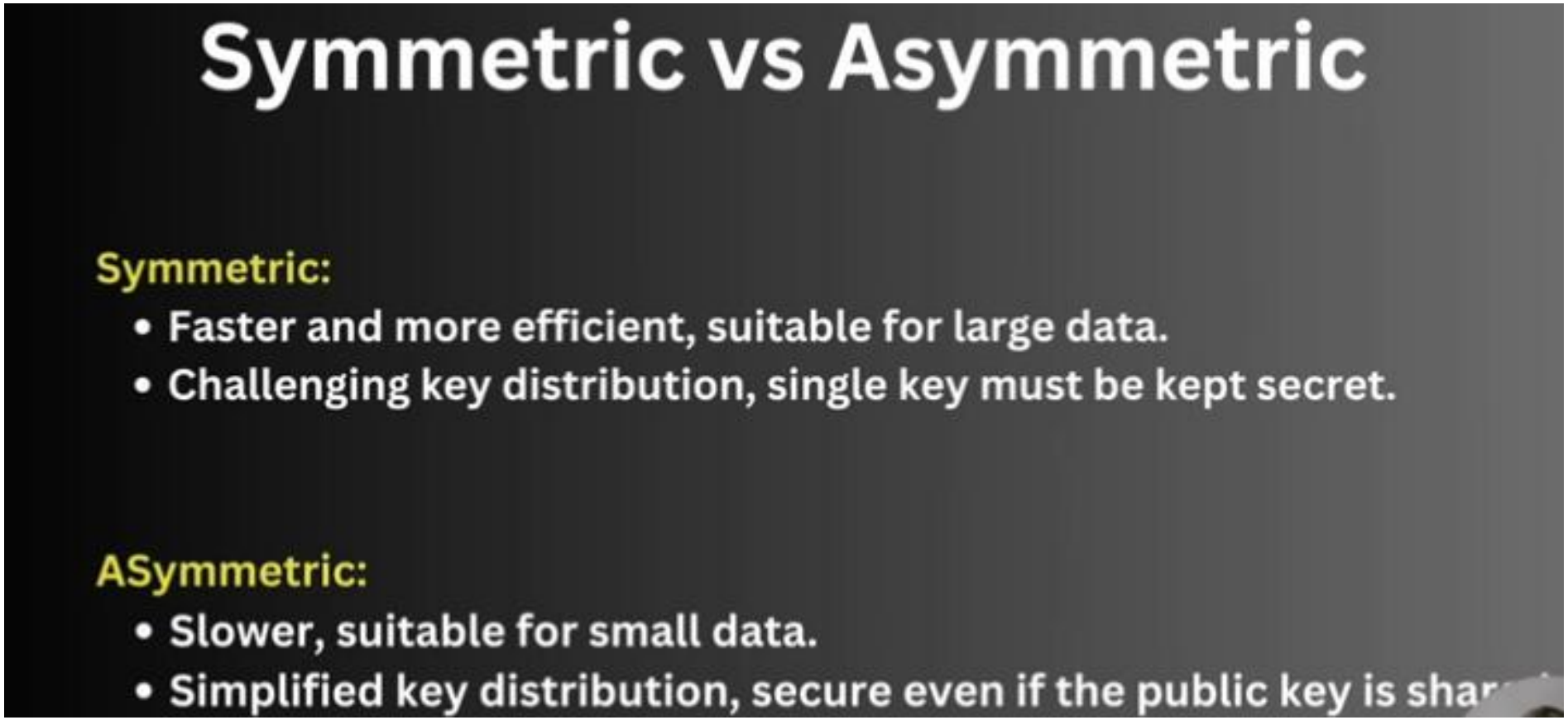
2. Server Shares
Public Key with Client



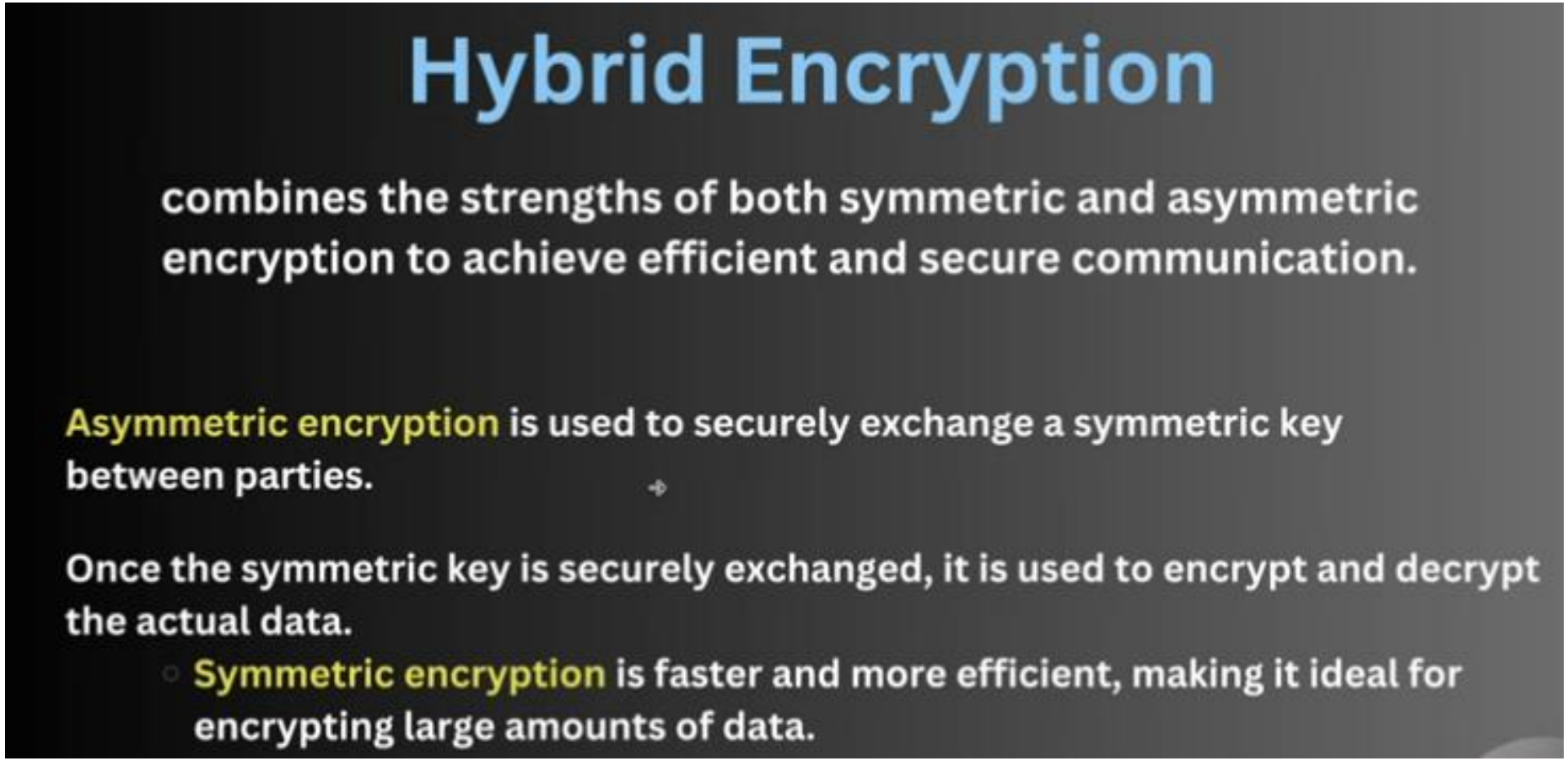
TLS


Symmetric vs Asymmetric
Symmetric:
• Faster and more efficient, suitable for large data.
• Challenging key distribution, single key must be kept secret.
ASymmetric:
• Slower, suitable for small data.
• Simplified key distribution, secure even if the public key is shar

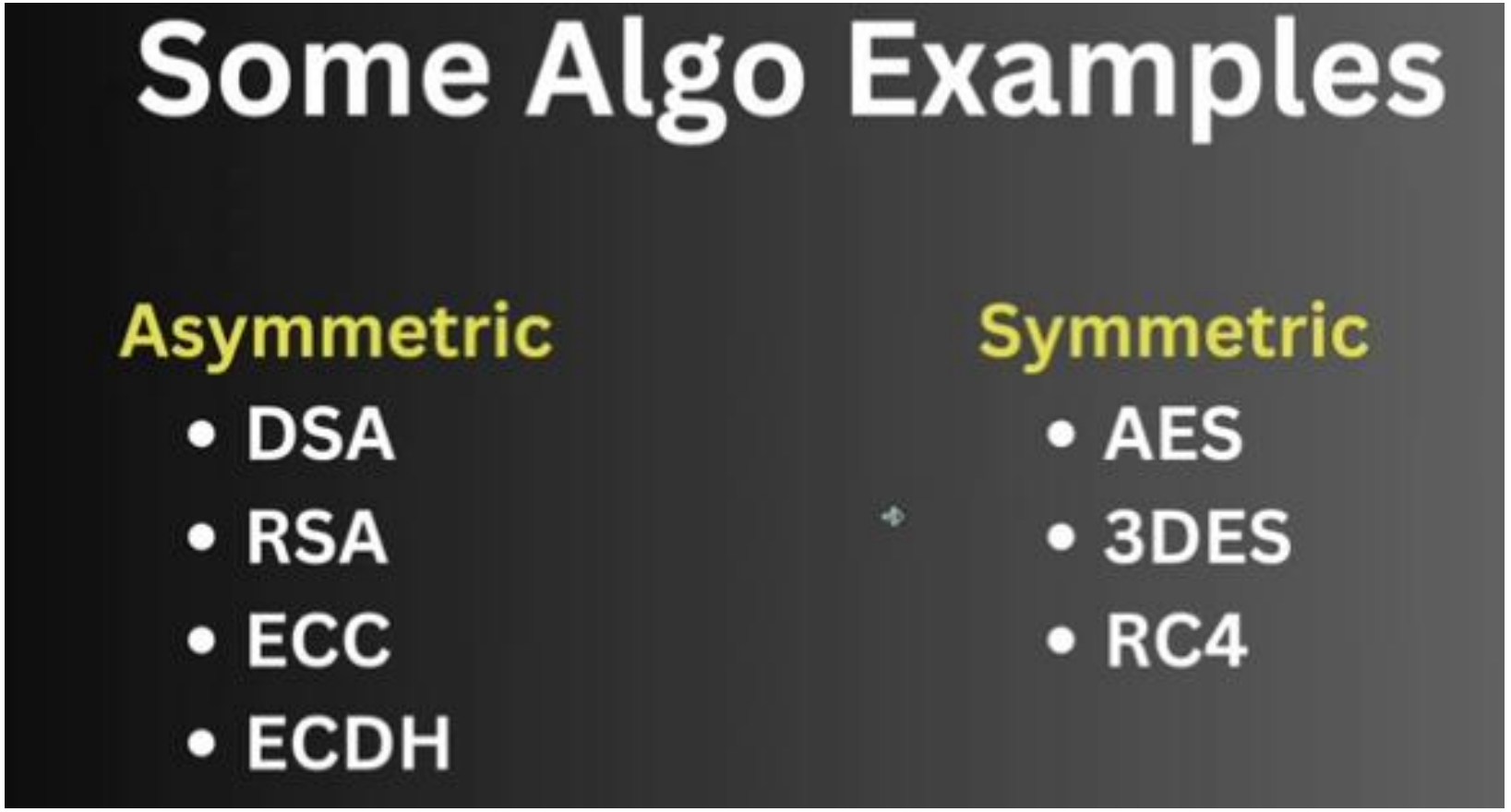
TLS


Symmetric vs Asymmetric
Symmetric:
• Faster and more efficient, suitable for large data.
• Challenging key distribution, single key must be kept secret.
ASymmetric:
• Slower, suitable for small data.
• Simplified key distribution, secure even if the public key is shar

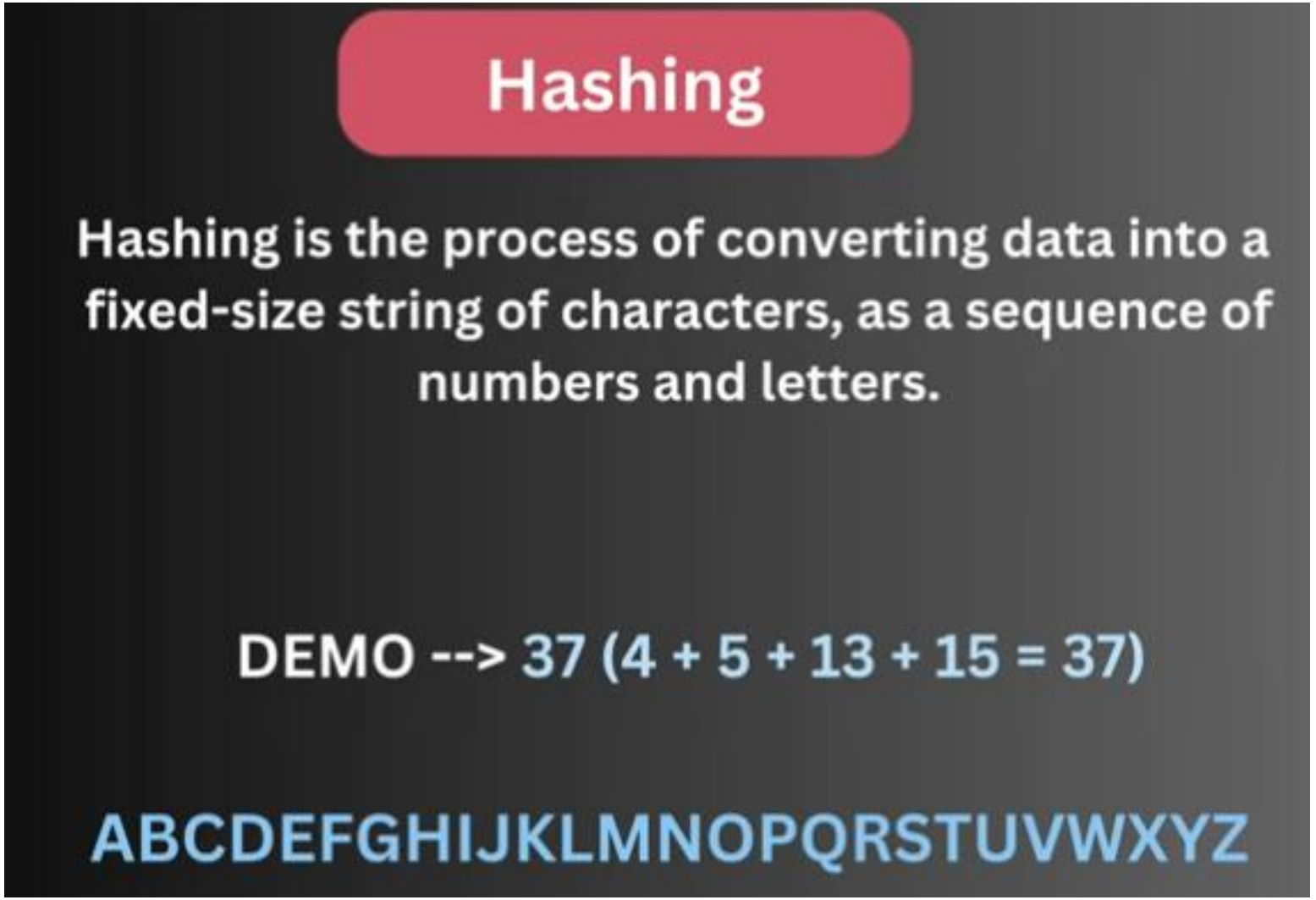
TLS


Hybrid Encryption
combines the strengths of both symmetric and asymmetric encryption to achieve efficient and secure communication.
Asymmetric encryption is used to securely exchange a symmetric key between parties.
Once the symmetric key is securely exchanged, it is used to encrypt and decrypt the actual data.
Symmetric encryption is faster and more efficient, making it ideal for encrypting large amounts of data.

TLS


Some Algo Examples
Asymmetric
• DSA
• RSA
• ECC
• ECDH
Symmetric
• AES
• 3DES
• RC4

TLS

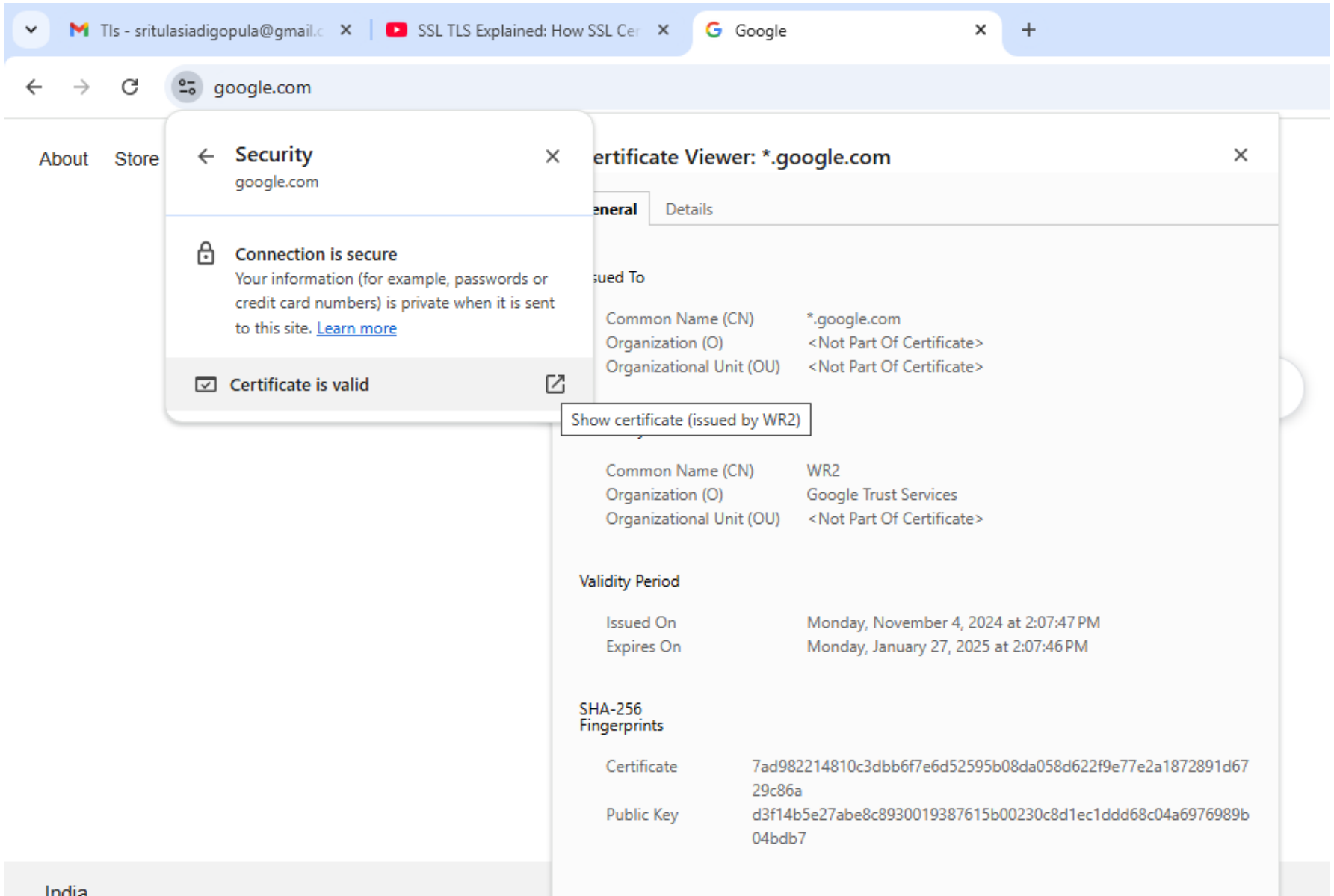

Hashing
Hashing is the process of converting data into a fixed-size string of characters, as a sequence of numbers and letters.
DEMO --> 37 (4 + 5 + 13 + 15 = 37)
ABCDEFGHIJKLMNOPQRSTUVWXYZ

TLS


Hashing
Most Common Hashing Algo
• MD5 (Message Digest Algorithm 5) (128bits)
• SHA (Secure Hash Algorithm)
◦ sha-1
◦ sha-2/3 224 256 384 512
Hash-based Message Authentication Code (sha256hmac)

TLS

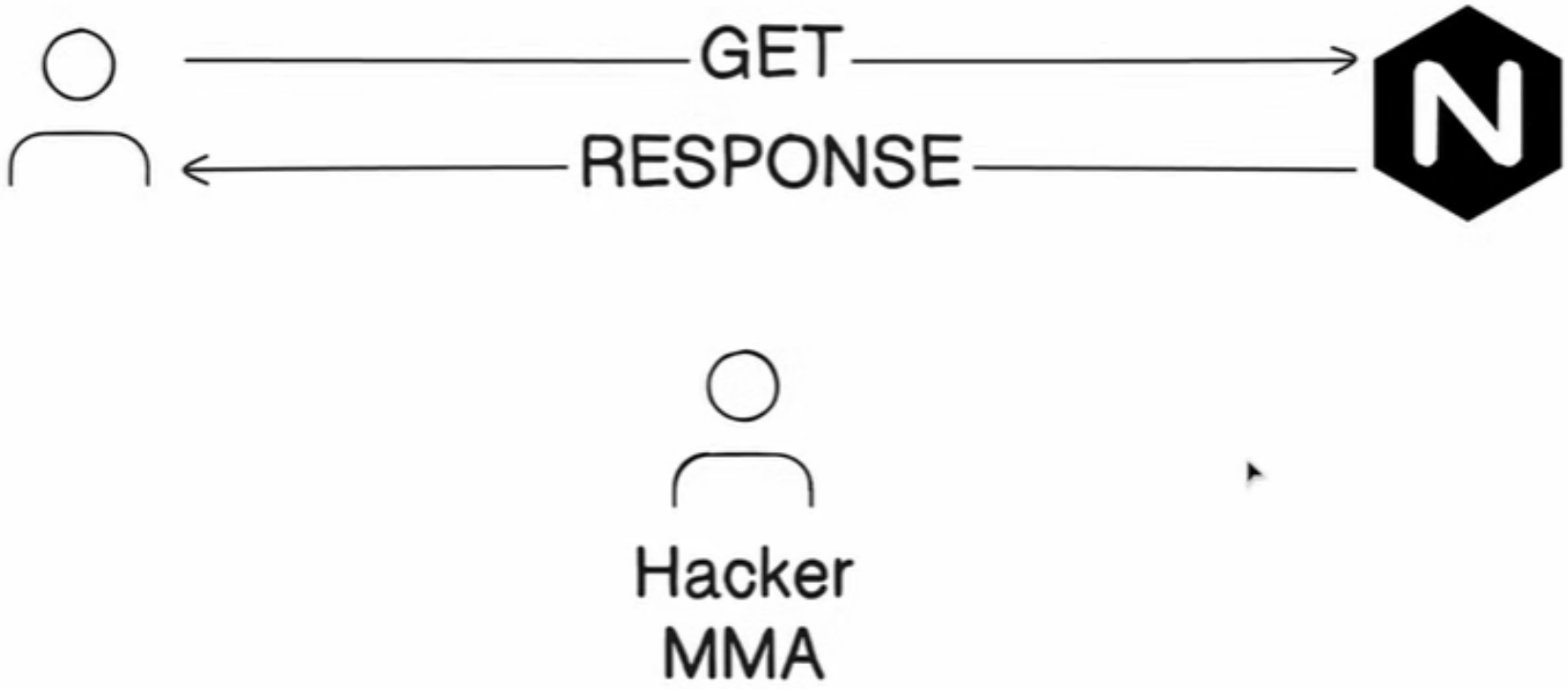
Authentication- Certificates



TLS



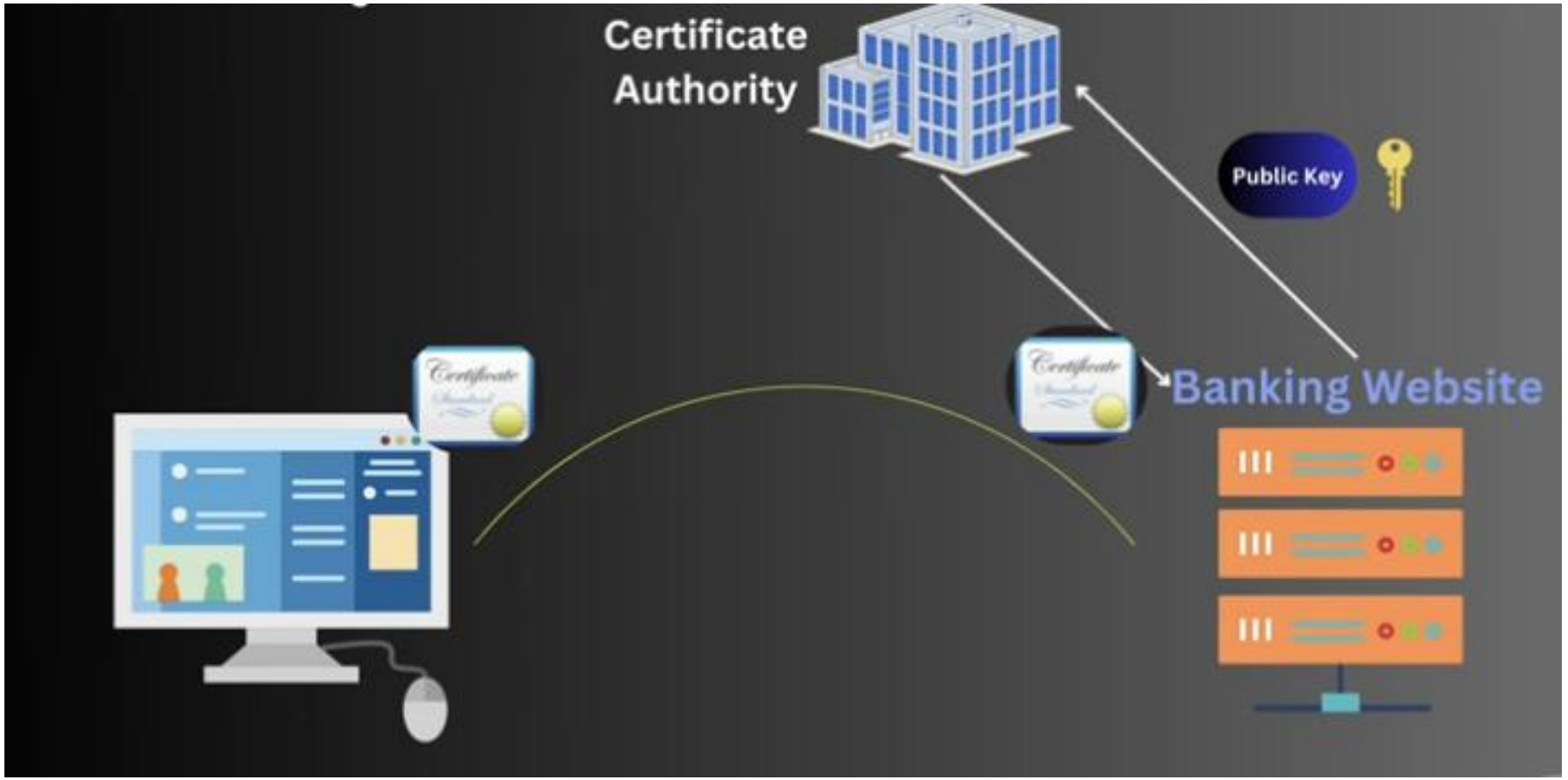
TLS-Need for certificates



TLS


A Certificate Authority (CA) is a trusted organization that issues digital certificates to verify the identity of websites and enable secure, encrypted communication over the internet.
CAs ensure the authenticity and integrity of the SSL certificates they provide.
Certificate
Standard

TLS



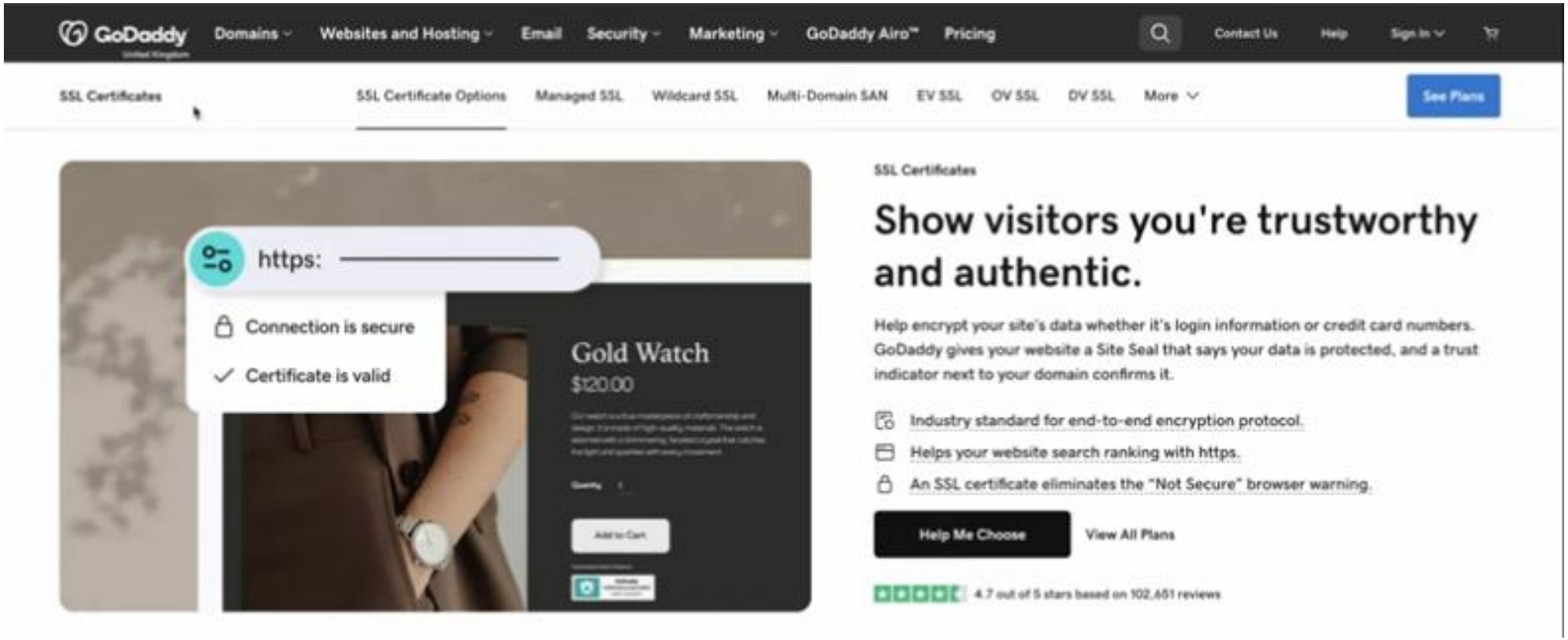
TLS

Formats for digital certificates

Format	Encoding	Common Extensions	Usage	Contains Private Key
PEM	Base64	.pem, .crt, .cer	Web servers, email	No (unless it's a key file)
DER	Binary	.der, .cer	Java platforms, binary data handling	No
PKCS#7	Base64 or Binary	.p7b, .p7c	Certificate chains	No
PKCS#12	Binary	.p12, .pfx	Export/import certs with keys	Yes

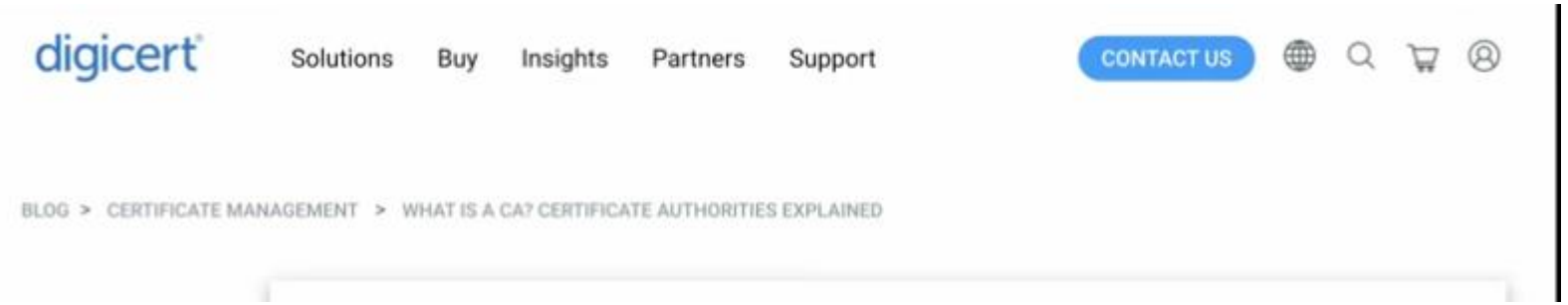
TLS

Certificate Providing Services



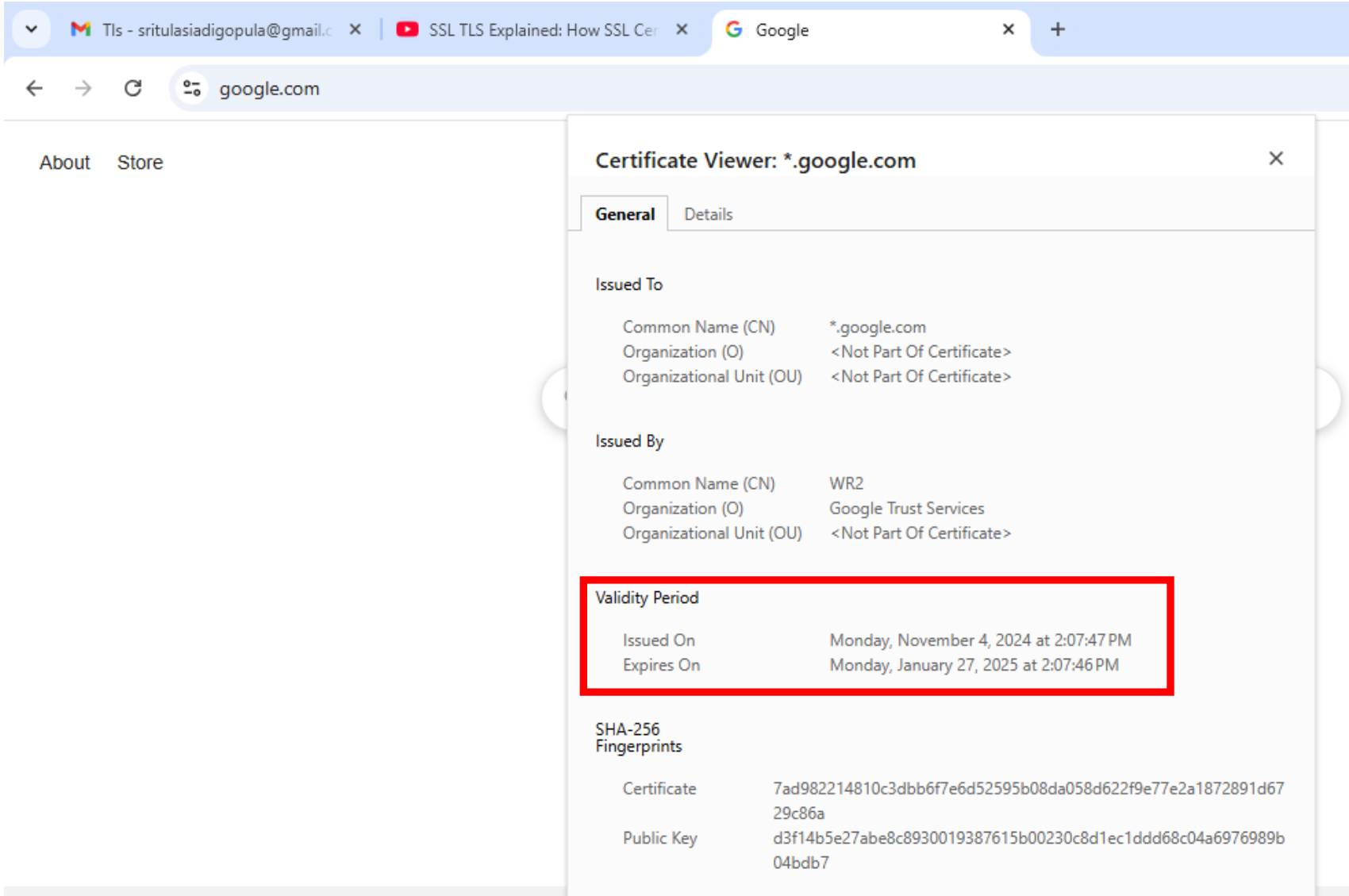
TLS

Certificate Providing Services



TLS

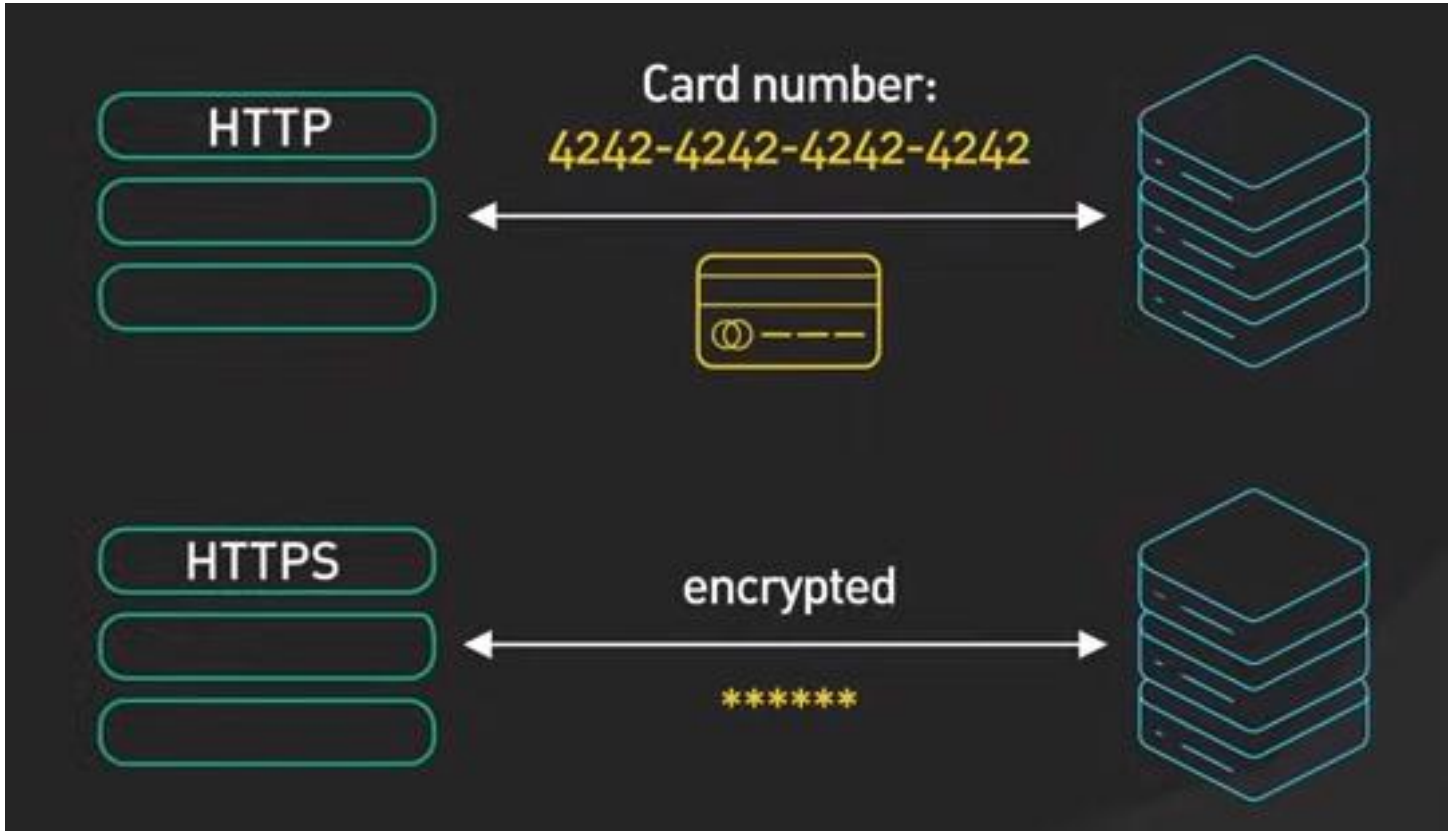
Validity for Certificates



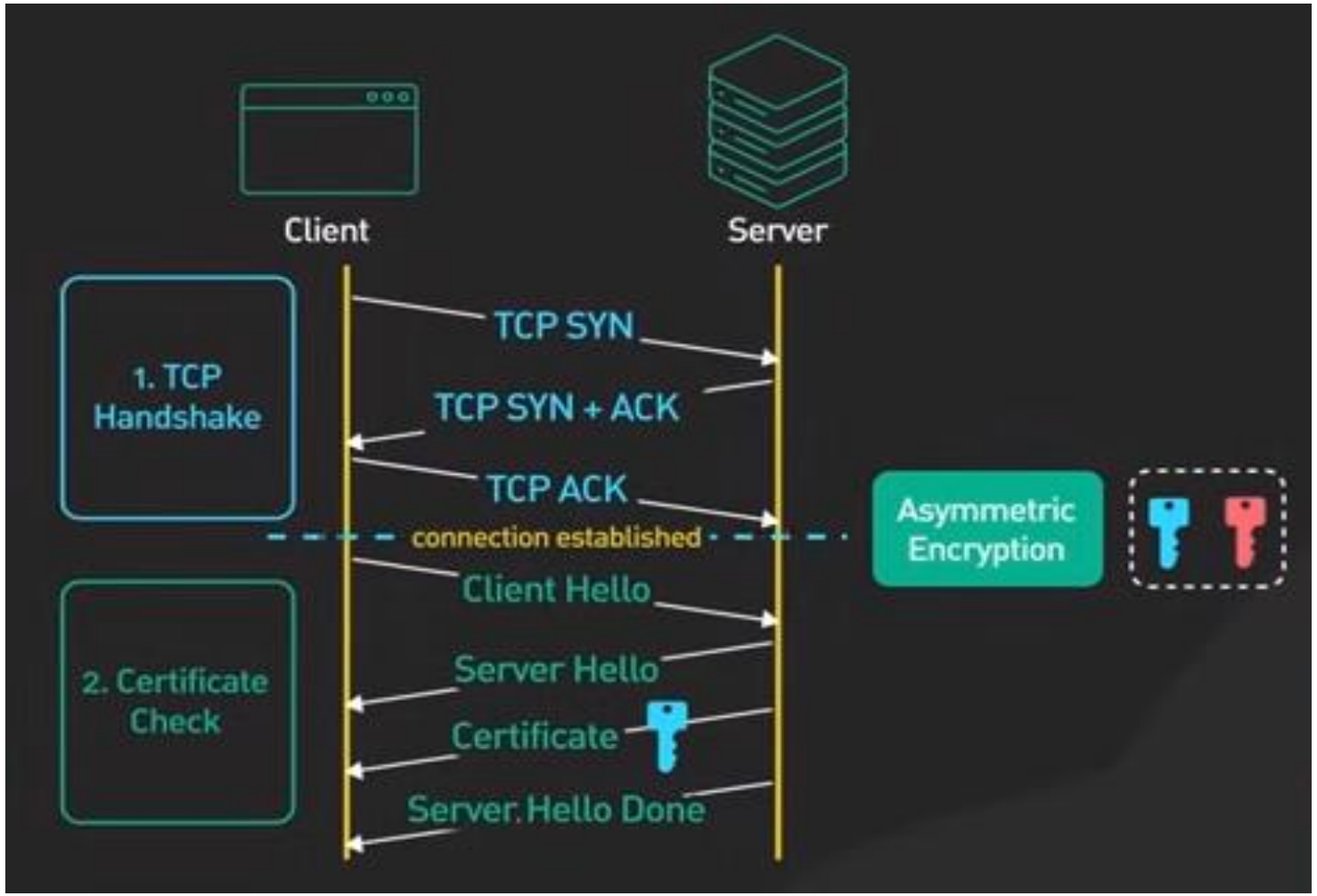
TLS

Let us see the TLS Connection Establishment Process now

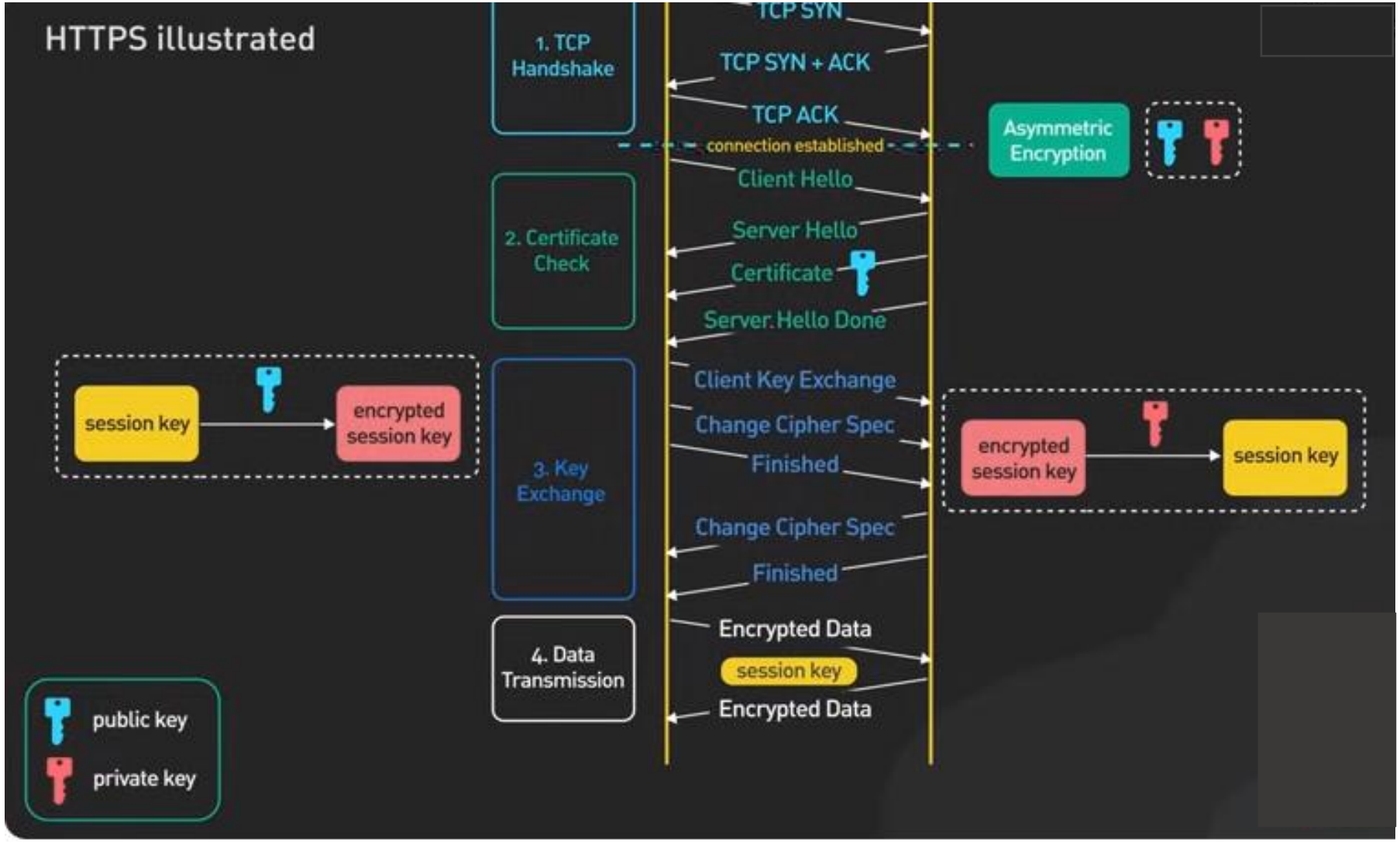
TLS



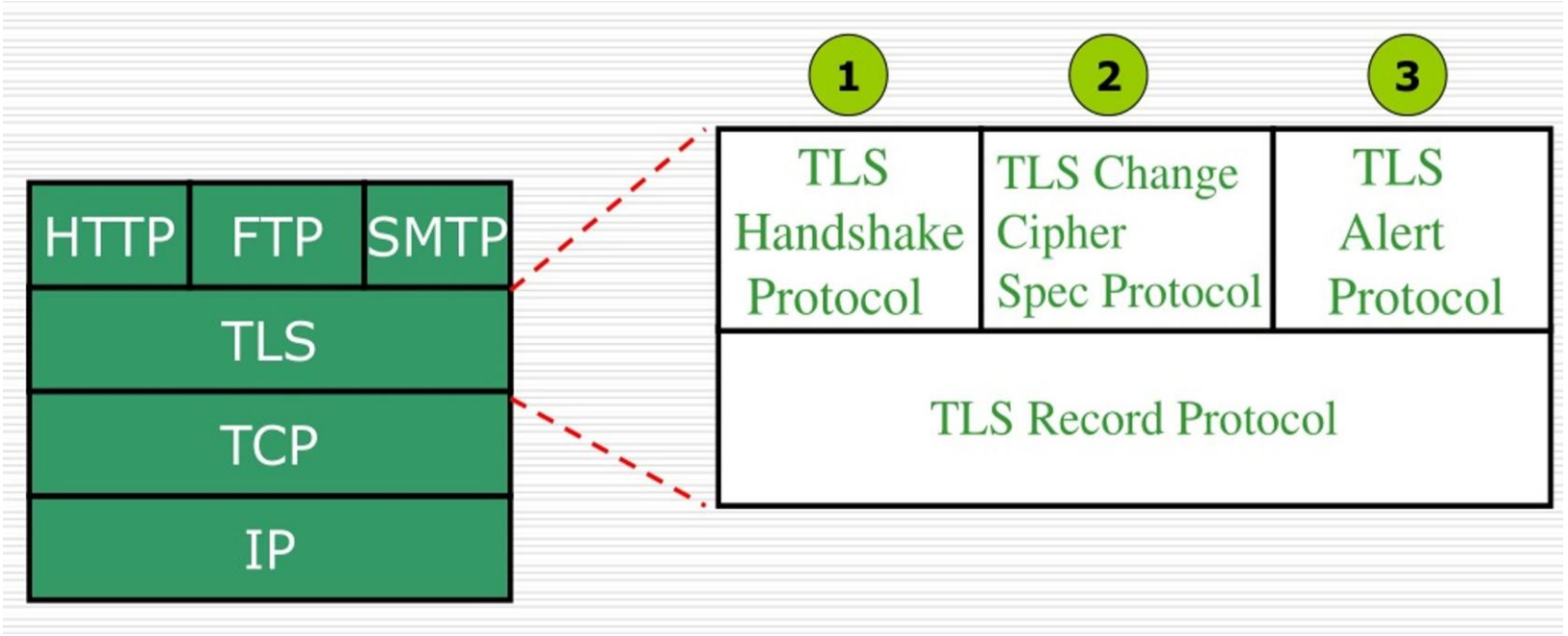
TLS


Client
Server
1. TCP Handshake
TCP SYN
TCP SYN + ACK
TCP ACK
connection established
Asymmetric Encryption
Client Hello
2. Certificate Check
Server Hello
Certificate
Server Hello Done

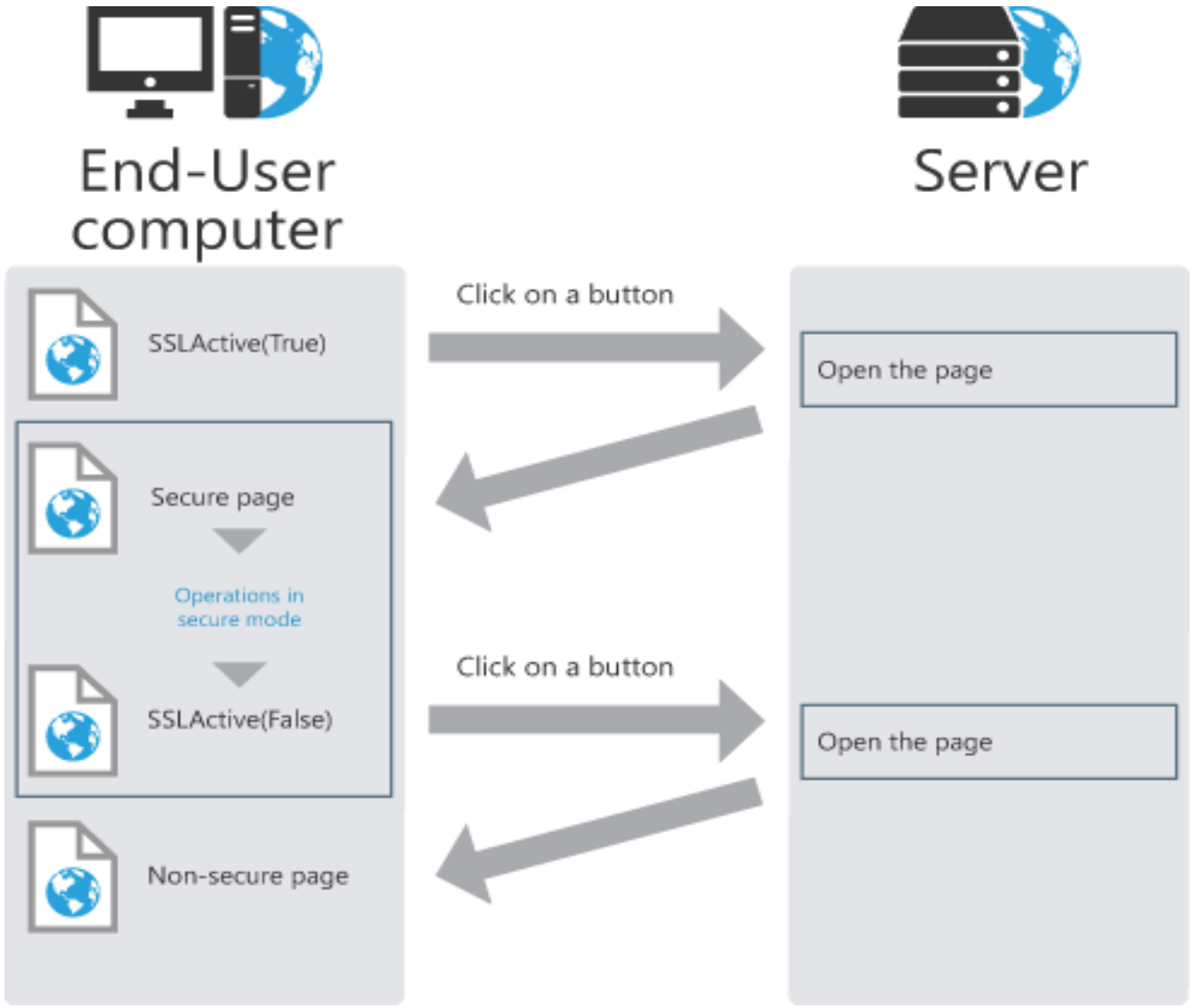
TLS


HTTPS illustrated
1. TCP Handshake
TCP SYN
TCP SYN + ACK
TCP ACK
connection established
Asymmetric Encryption
Client Hello
2. Certificate Check
Server Hello
Certificate
Server.Hello Done
session key
encrypted session key
3. Key Exchange
Client Key Exchange
Change Cipher Spec
Finished
encrypted session key
session key
Change Cipher Spec
Finished
4. Data Transmission
Encrypted Data
session key
Encrypted Data
public key
private key

TLS-Protocol Stack

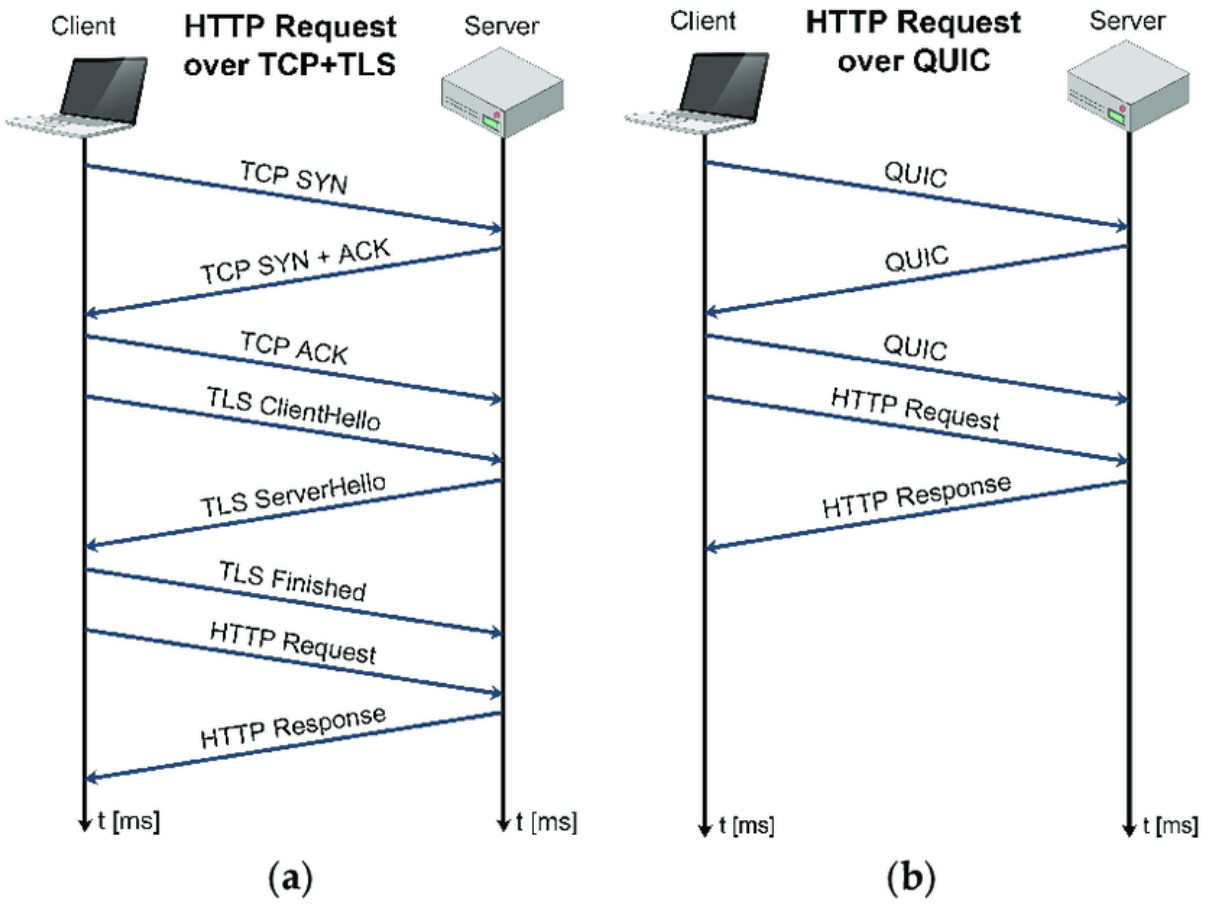


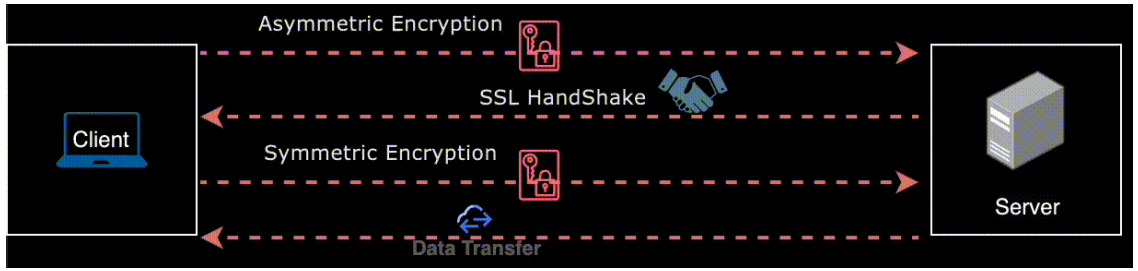
TLS


End-User computer
Server
SSLActive(True)
Click on a button
Open the page
Secure page
Operations in secure mode
SSLActive(False)
Click on a button
Open the page
Non-secure page

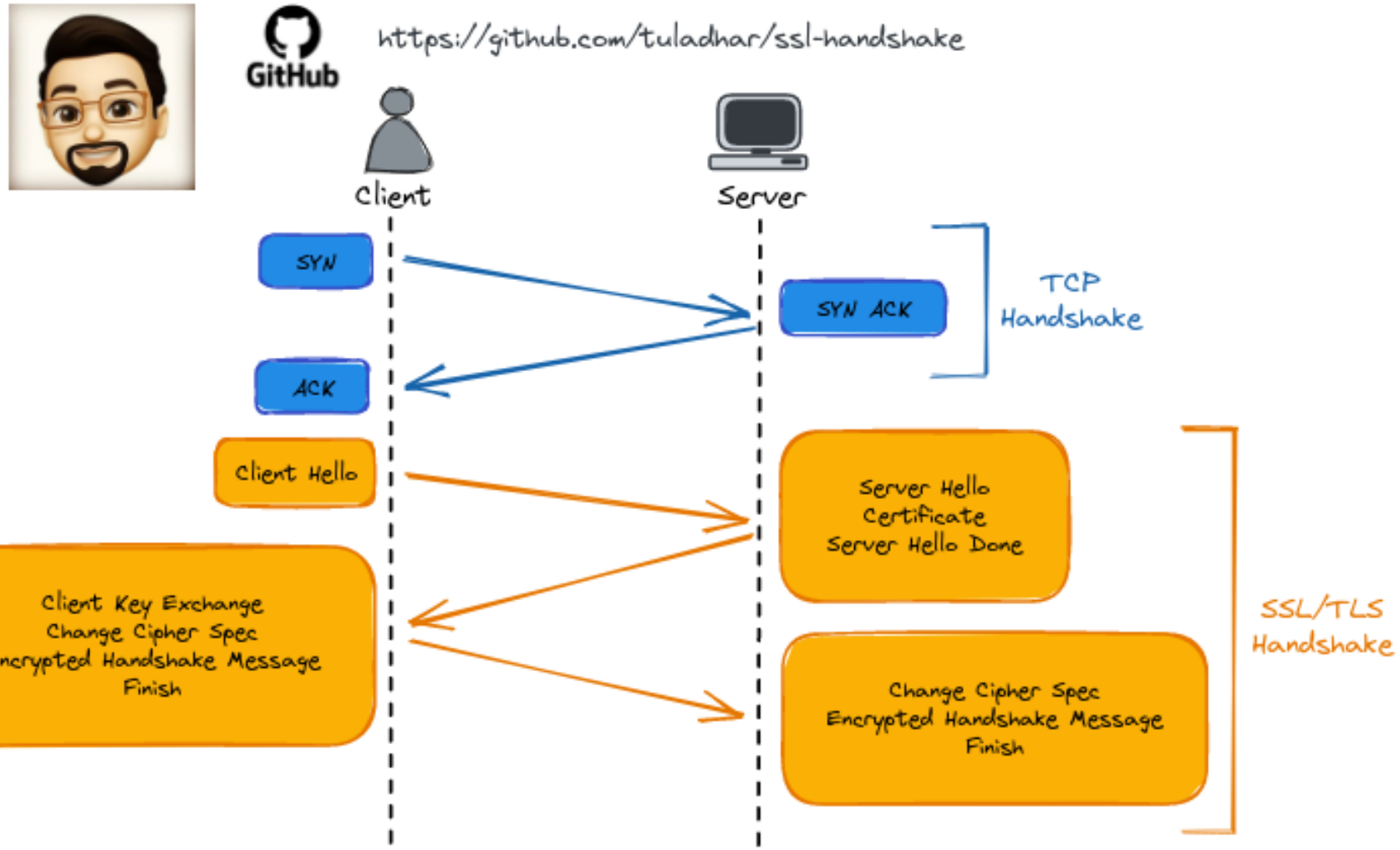

What are SSL/TLS Certificates???

TLS


Client
HTTP Request over TCP+TLS
Server
TCP SYN
TCP SYN + ACK
TCP ACK
TLS ClientHello
TLS ServerHello
TLS Finished
HTTP Request
HTTP Response
t [ms]
t [ms]
(a)
Client
HTTP Request over QUIC
Server
QUIC
QUIC
QUIC
HTTP Request
HTTP Response
t [ms]
t [ms]
(b)


Asymmetric Encryption
SSL HandShake
Client
Symmetric Encryption
Server
Data Transfer

TLS


GitHub
https://github.com/tuladhar/ssl-handshake
Client
Server
SYN
SYN ACK
TCP Handshake
ACK
Client Hello
Server Hello
Certificate
Server Hello Done
Client Key Exchange
Change Cipher Spec
Encrypted Handshake Message
Finish
Change Cipher Spec
Encrypted Handshake Message
Finish
SSL/TLS Handshake

TLS

Features of TLS

- **Encryption**:
  - Protects data from being intercepted or viewed by unauthorized parties.
  - Ensures privacy during communication.
- **Integrity**:
  - Uses cryptographic hash functions (e.g., SHA-2) to prevent tampering with data during transmission.
- **Authentication**:
  - Verifies the identity of parties involved, often through digital certificates issued by trusted Certificate Authorities (CAs).
- **Session Management**:
  - Supports secure key exchange to establish a session key for encrypting data.
  - Enables efficient communication through session resumption techniques.

TLS

How TLS Works with the Transport Layer

- **TLS and TCP**:
  - TLS relies on **TCP** to provide reliable data transmission.
  - The TCP layer handles packet delivery, while TLS encrypts and secures the transmitted data.
- **Connection Establishment**:
  - A **TLS handshake** is performed after the TCP connection is established:
    - Negotiates encryption algorithms.
    - Exchanges keys using methods like RSA or Diffie-Hellman.
    - Authenticates the server and optionally the client.
  - After the handshake, all communication is encrypted.
- **Layer Placement**:
  - TLS operates between the transport layer (e.g., TCP) and the application layer (e.g., HTTPS, IMAP, or SMTP).
  - Applications like web browsers, email clients, and messaging apps use TLS to secure data before sending it to TCP.

TLS

Use Cases of TLS

- **Web Security (HTTPS)**:
  - Secures data exchanged between web browsers and servers.
- **Email Communication**:
  - Used in protocols like SMTP, IMAP, and POP3 for secure email transmission.
- **File Transfers**:
  - Protocols like FTPS (FTP Secure) use TLS to encrypt file transfers.
- **Virtual Private Networks (VPNs)**:
  - Some VPNs use TLS for secure tunnelling.
- **VoIP and Messaging**:
  - Protects voice and instant messaging communications.

TLS

Advantages of TLS

- **Strong Security**:
  - Provides robust encryption and authentication mechanisms.
- **Wide Adoption**:
  - Supported by major protocols and applications, ensuring interoperability.
- **Flexible Cryptographic Algorithms**:
  - Supports a range of cipher suites for encryption, hashing, and key exchange.

TLS

Disadvantages of TLS

- **Performance Overhead**:
  - Encryption and decryption can increase CPU usage and latency.
- **Complexity**:
  - Misconfiguration can weaken security (e.g., using outdated cipher suites).
- **Resource Intensive**:
  - TLS handshakes and cryptographic operations consume additional bandwidth and memory.

TLS

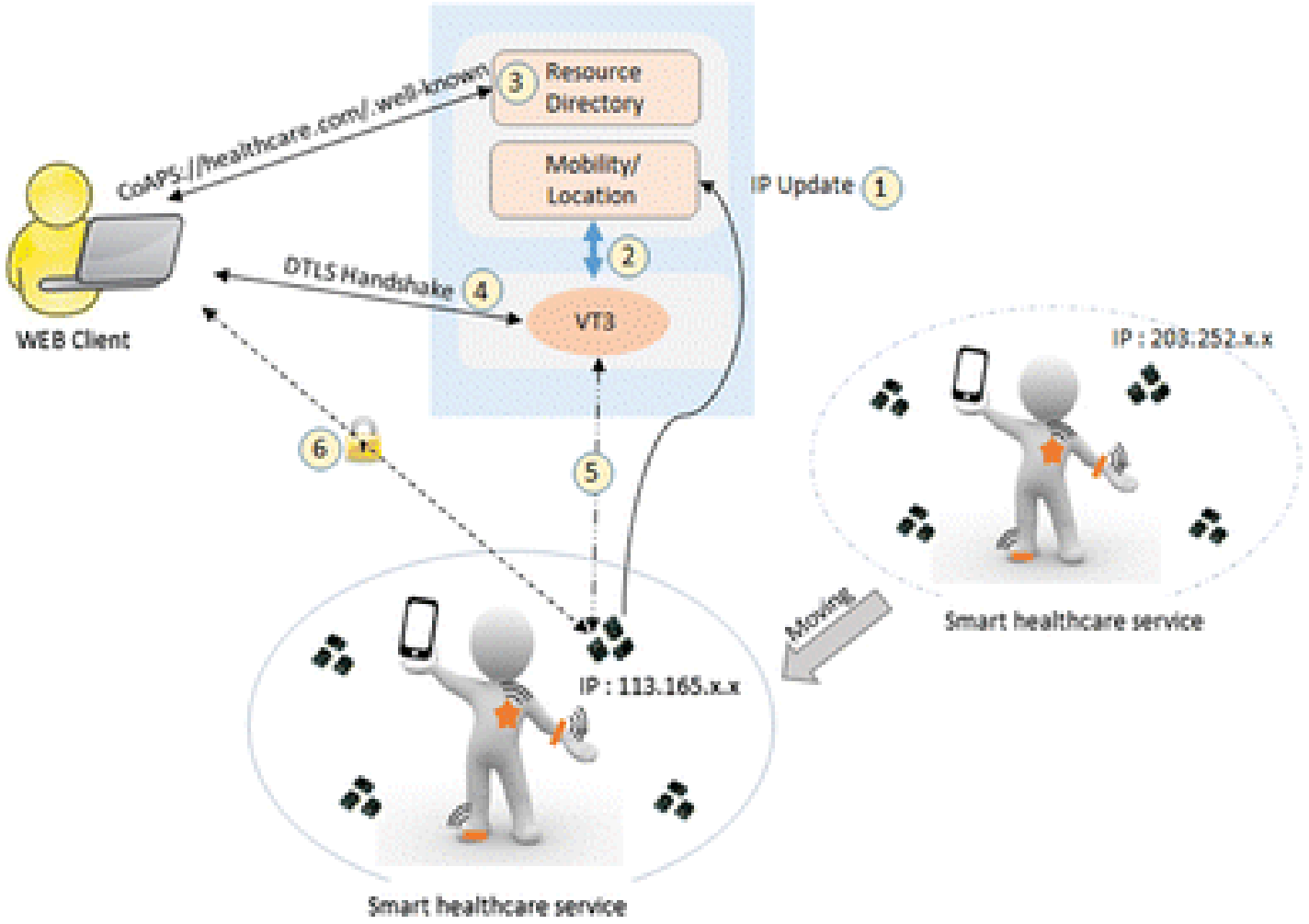
- TLS enhances security at the transport layer by encrypting data, verifying authenticity, and ensuring integrity.
- While it relies on TCP for reliable data transfer, it significantly strengthens the overall security of network communications, making it a cornerstone of modern internet security protocols like HTTPS.

DTLS

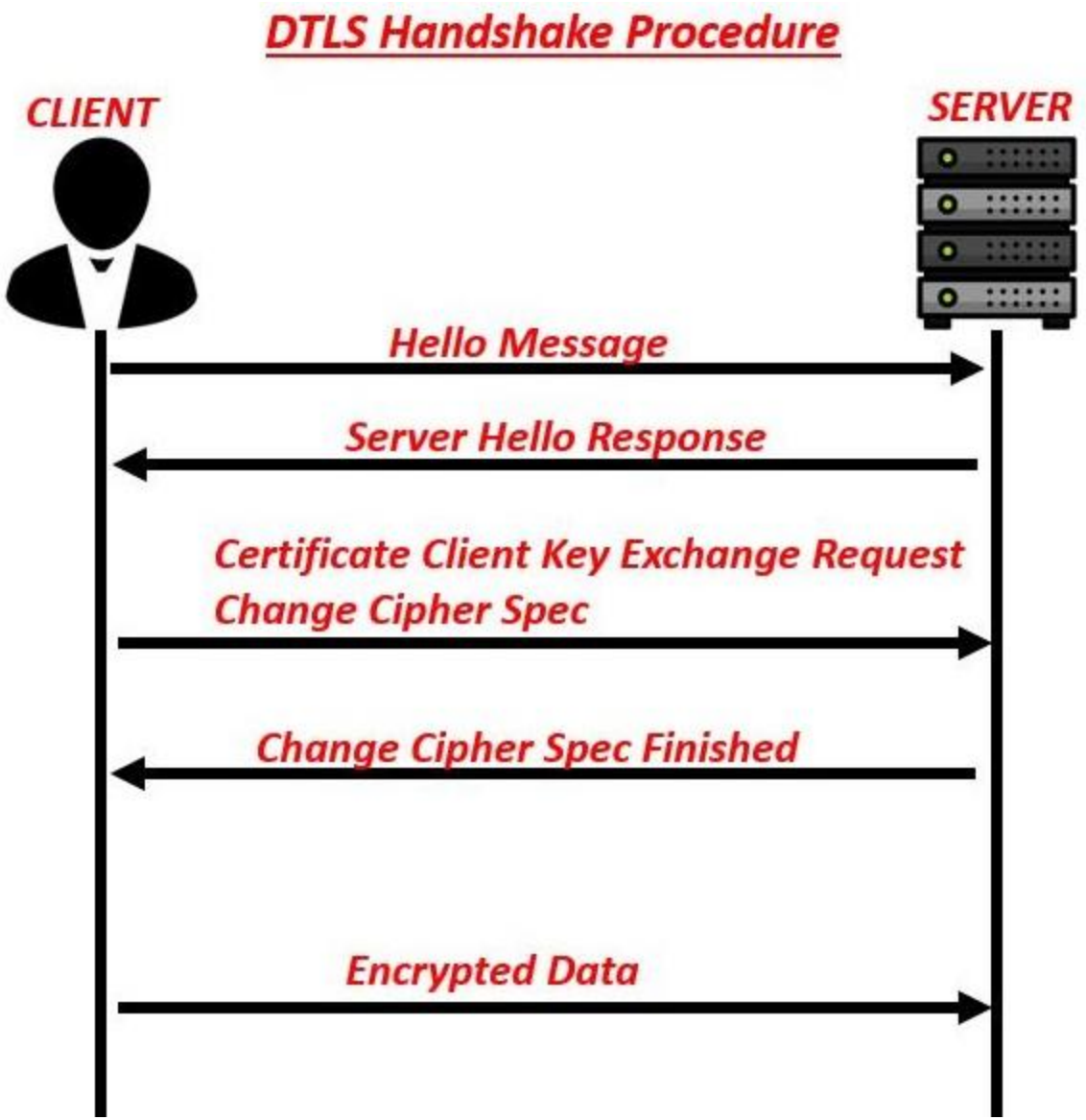
DTLS (Datagram Transport Layer Security)

- **DTLS** is a variant of **TLS (Transport Layer Security)** designed to provide security for **datagram-based** protocols like **UDP** (User Datagram Protocol).
- While TLS operates over connection-oriented protocols (like TCP), **DTLS** is specifically tailored for **connectionless** protocols such as UDP, which do not guarantee reliability, ordering, or error correction.
- DTLS aims to provide the same security services as TLS, such as encryption, integrity, and authentication, but for **real-time, low-latency applications** that require the speed of UDP.

DTLS


CoAPS://healthcare.com/.well-known
3
Resource Directory
Mobility/ Location
IP Update
1
2
DTLS Handshake
4
VT3
WEB Client
6
5
IP : 203.252.x.x
Smart healthcare service
Moving
IP : 113.165.x.x
Smart healthcare service

DTLS


DTLS Handshake Procedure
CLIENT
SERVER
Hello Message
Server Hello Response
Certificate Client Key Exchange Request
Change Cipher Spec
Change Cipher Spec Finished
Encrypted Data

DTLS

Key Features of DTLS

- **Encryption**:
  - Like TLS, DTLS provides **confidentiality** by encrypting the data exchanged between parties, preventing eavesdropping.
- **Integrity**:
  - DTLS ensures **data integrity** by including message authentication codes (MACs) to detect any changes to the transmitted data.
- **Authentication**:
  - DTLS can authenticate the identities of the communicating parties using **digital certificates** (issued by trusted Certificate Authorities).
- **Replay Protection**:
  - DTLS includes mechanisms to prevent attackers from replaying previously captured messages to the recipient (replay attacks).
- **Datagram Support**:
  - Designed for protocols that do not guarantee delivery or ordering, DTLS ensures security on top of UDP's connectionless nature.
- **Session Resumption**:
  - DTLS can reuse session keys from previous connections to reduce the overhead of renegotiating security parameters.

DTLS

How DTLS Works

DTLS works similarly to TLS but adapts to the challenges of UDP:

1.Handshake:

1. The handshake process in DTLS is very similar to TLS and involves negotiating cryptographic parameters and securely exchanging keys. However, the handshake is designed to tolerate packet loss, reordering, and out-of-order delivery, which are common in UDP networks.

2.Packet Ordering:

1. Unlike TLS, which operates over TCP (which ensures packet ordering), DTLS must handle situations where packets may arrive out of order. To address this, DTLS includes **sequence numbers** to detect and discard out-of-order packets.

3.Reliability Concerns:

1. Since UDP does not provide reliability or retransmission, DTLS handles issues like packet loss or duplication by using **time stamps** and **sequence numbers**.

4.No Connection State:

1. DTLS operates without maintaining a persistent connection state, aligning with UDP's connectionless nature.

DTLS

Applications of DTLS

DTLS is used in scenarios where low-latency communication and high throughput are needed, and where the overhead of establishing and maintaining a TCP connection (as in TLS) is not desirable:

- **VoIP (Voice over IP)**:
  - Used in protocols like **SRTP (Secure Real-Time Protocol)** to secure voice communication over UDP.
- **Video Streaming**:
  - DTLS can secure real-time video streams that use UDP (e.g., in some video conferencing systems).
- **Online Gaming**:
  - Multiplayer games that rely on low-latency, high-speed communication often use DTLS for securing data exchanged over UDP.
- **VPNs**:
  - **DTLS** is used in VPNs like **OpenVPN** for securing datagrams transmitted over UDP. It ensures that packets are encrypted and integrity-checked while benefiting from the performance advantages of UDP.
- **IoT (Internet of Things)**:
  - Many IoT devices use UDP due to its low overhead and DTLS to ensure secure communication.
- **Datagram-based Applications**:
  - Secure communication for applications using **QUIC** or similar datagram-based protocols.

DTLS

Advantages of DTLS

1.Low Latency:

1. DTLS is designed for low-latency applications, making it ideal for real-time communication.

2.UDP Benefits:

1. It provides the security benefits of TLS while maintaining the efficiency and lower overhead of UDP.

3.Compatibility with UDP:

1. DTLS allows secure communication over UDP, which is typically used in applications requiring speed and minimal delay.

4.Flexibility:

1. DTLS can be used in a variety of applications that need secure communication but cannot rely on the connection-oriented nature of TCP.

DTLS

Disadvantages of DTLS

1.No Built-in Reliability:

1. Since DTLS operates over UDP, it does not guarantee delivery or ordering. Applications need to handle lost or out-of-order packets themselves.

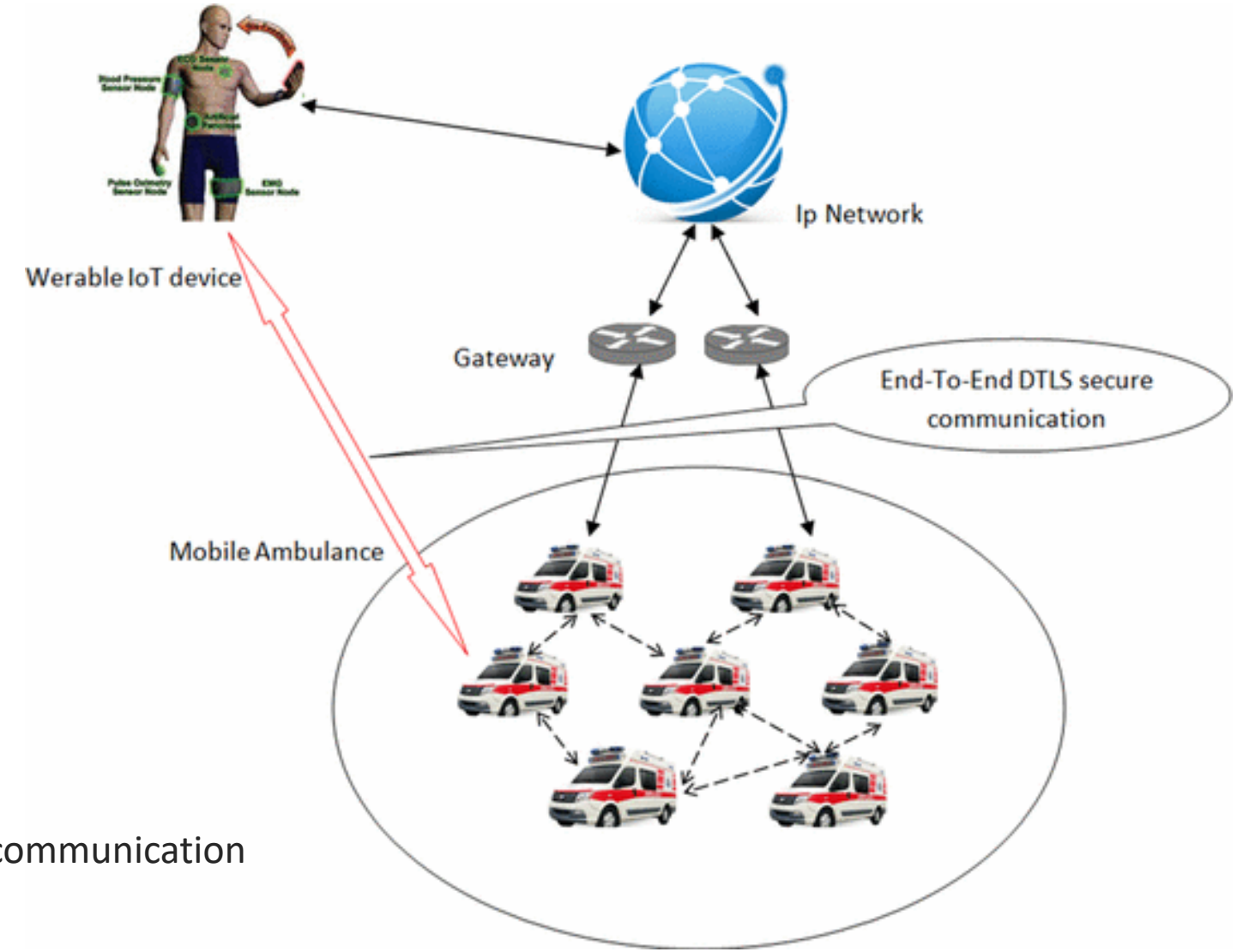
2.Overhead:

1. While it reduces the overhead compared to TLS, the encryption and decryption processes still add some computational cost.

3.Complexity:

1. The DTLS handshake can be more complicated due to the need to handle packet loss and reordering, which is less straightforward than the TLS handshake over TCP.

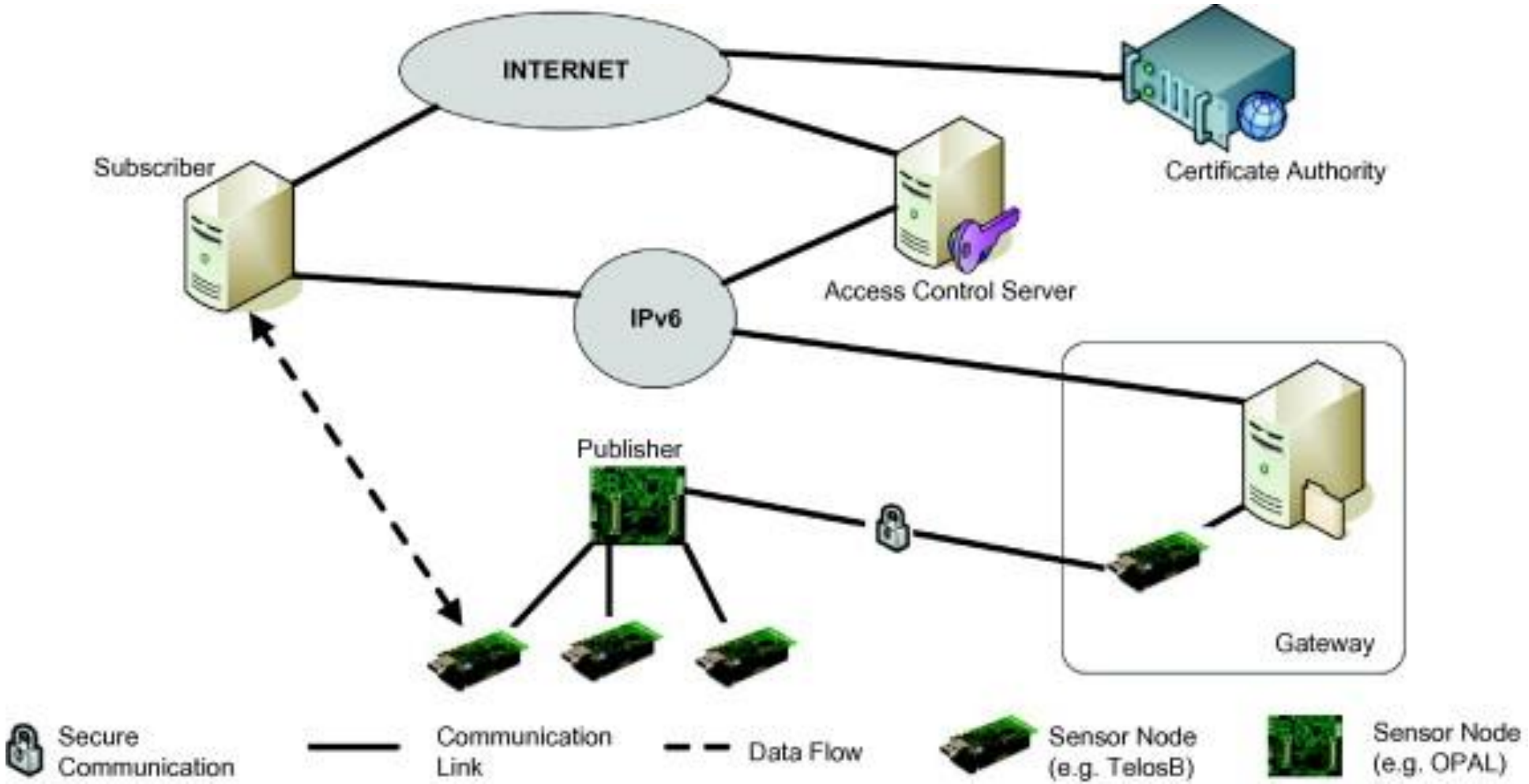
DTLS



DTLS end-to-end secure communication

DTLS

DTLS based security and two-way authentication for the Internet of Things



DTLS

DTLS based security and two-way authentication for the Internet of Things

Feature	TLS	DTLS
Underlying Protocol	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Reliability	Built-in; uses TCP's reliability features	Adds its own mechanisms for reliability
Connection	Connection-oriented	Connectionless
Use Case	Reliable data transmission (e.g., HTTPS, email)	Low-latency or real-time communication (e.g., VoIP, online gaming)
Order Guarantee	Ensures ordered delivery of data	Adds optional mechanisms to handle reordering

DTLS

DTLS based security and two-way authentication for the Internet of Things

Handshake Overhead	Lower, due to TCP's reliable nature	Slightly higher, as it handles retransmissions and ordering
Packet Loss Handling	Handled by TCP	Handles packet loss within the protocol
Streaming	Well-suited for streaming data over reliable connections	Suitable for unreliable, real-time data streams
Security	Same security guarantees	Similar guarantees, with added resilience to UDP-specific challenges

Why Transport-Layer Security Matters in IoT

Constrained devices, wireless links, and open networks are exposed at every hop — transport security is the last line of defense.

C

Confidentiality

Encrypt sensor data & commands so eavesdroppers on the wireless link learn nothing.

I

Integrity

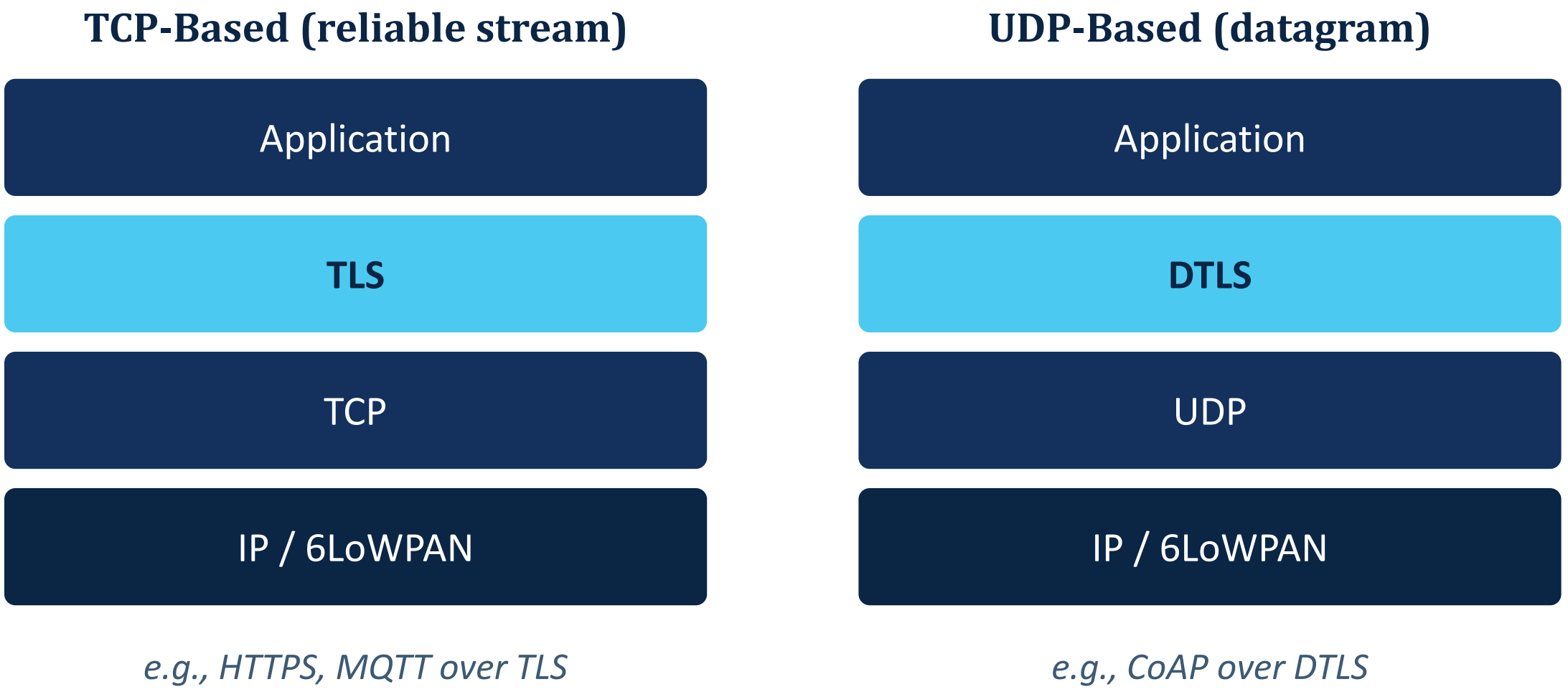
Detect any tampering with data in transit using MACs or AEAD authentication tags.

A

Authentication

Prove device and server identity to stop spoofed nodes and rogue gateways.

Where TLS and DTLS Fit in the IoT Stack



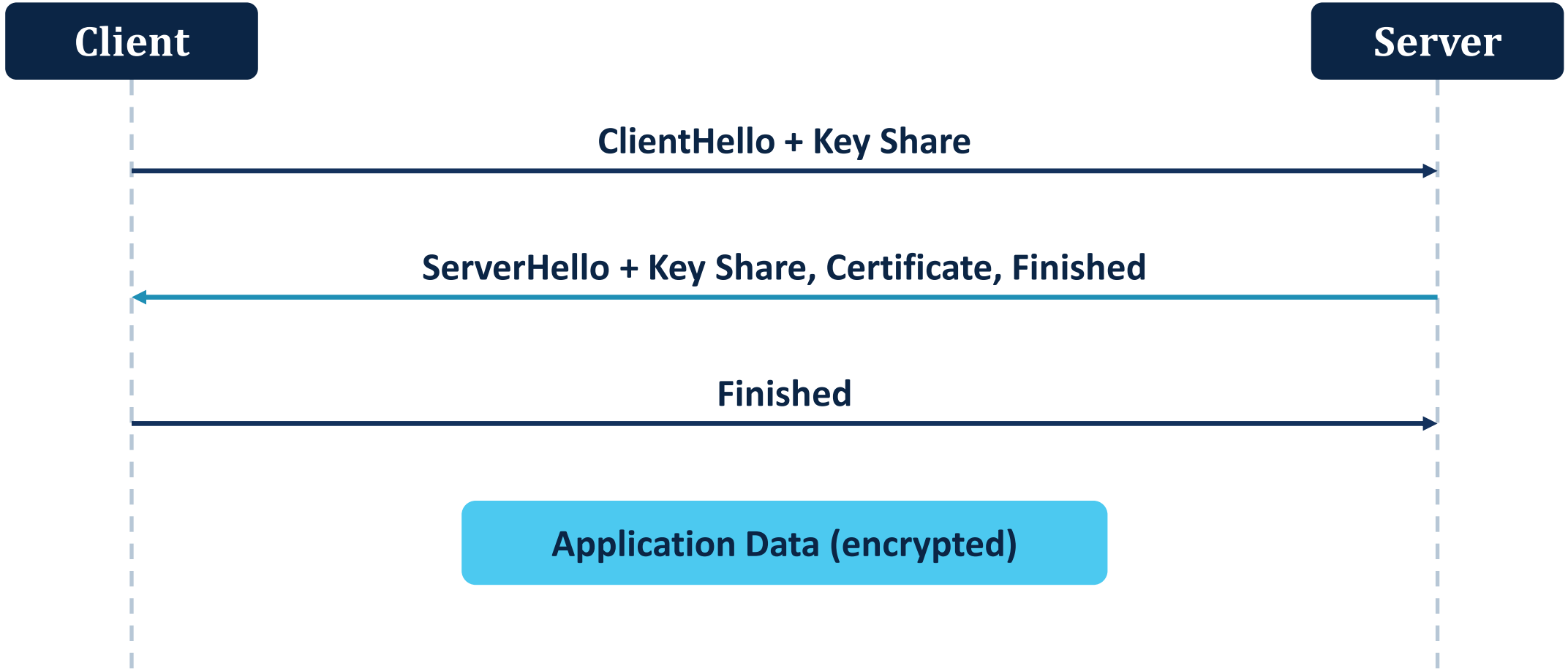
TLS: Transport Layer Security — Overview

- Runs on top of a reliable, ordered byte stream — always TCP
- Delivers confidentiality (encryption), integrity (AEAD/MAC) and authentication (certificates)
- Current standard is TLS 1.3 (RFC 8446) — fewer round trips, stronger defaults than 1.2
- Secures HTTPS, MQTT over TLS (port 8883), and CoAP over TLS



TLS 1.3 secures the connection

TLS 1.3 Handshake — One Round Trip



1-RTT handshake — encrypted traffic begins immediately after

TLS Record Protocol — Framing the Data



- Type — identifies handshake, alert, or application-data content
- Version — legacy field, fixed for compatibility
- Length — size of the record payload (max 2^{14} bytes)
- Encrypted payload — AEAD-protected data plus its authentication tag

TLS Challenges on Constrained IoT Devices

1 TCP Dependency

Connection setup and congestion control add latency and overhead on lossy radio links.

2 Handshake Cost

Multiple round trips and certificate exchange strain low-power radios like IEEE 802.15.4.

3 Memory & Compute

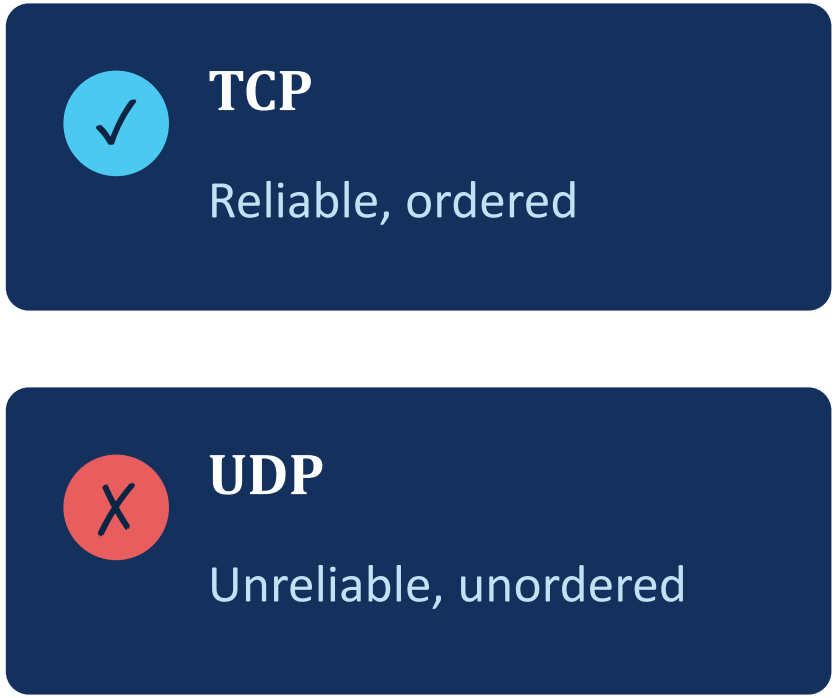
Full TLS stacks (certificate parsing, RSA/ECC math) can exceed KB-scale RAM and ROM budgets.

4 Energy Drain

Every extra byte transmitted and every round trip shortens battery-powered node lifetime.

DTLS: Datagram Transport Layer Security

- TLS assumes a reliable, in-order, connection-oriented transport — TCP
- Many IoT protocols run over UDP: connectionless, unordered, tolerant of loss
- DTLS (RFC 6347; DTLS 1.3 = RFC 9147) adapts TLS's security to unreliable datagrams
- Same cryptographic guarantees as TLS, plus mechanisms to cope with loss & reordering



TLS vs. DTLS — Key Differences

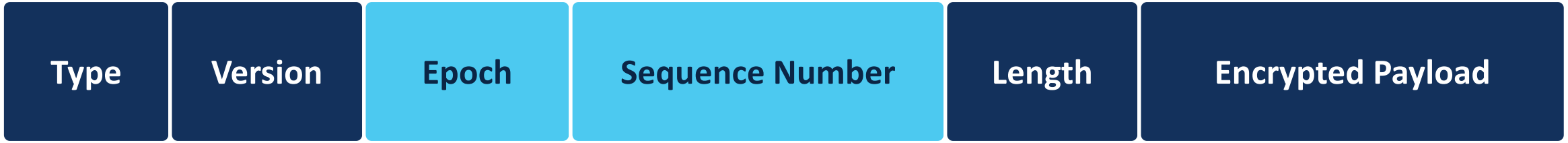
Aspect	TLS	DTLS
Transport	TCP	UDP
Reliability	Provided by TCP	Handled by DTLS (retransmit timers)
Ordering	Guaranteed by TCP	Explicit epoch + sequence number
Anti-DoS step	Not needed	Stateless cookie exchange
Fragmentation	Rare (byte stream)	Required to fit datagram MTU
Typical use	HTTPS, MQTT	CoAP, telemetry, multicast

DTLS Handshake — Coping with an Unreliable Link



Cookie stops spoofed-source DoS • lost flights are retransmitted after a timeout

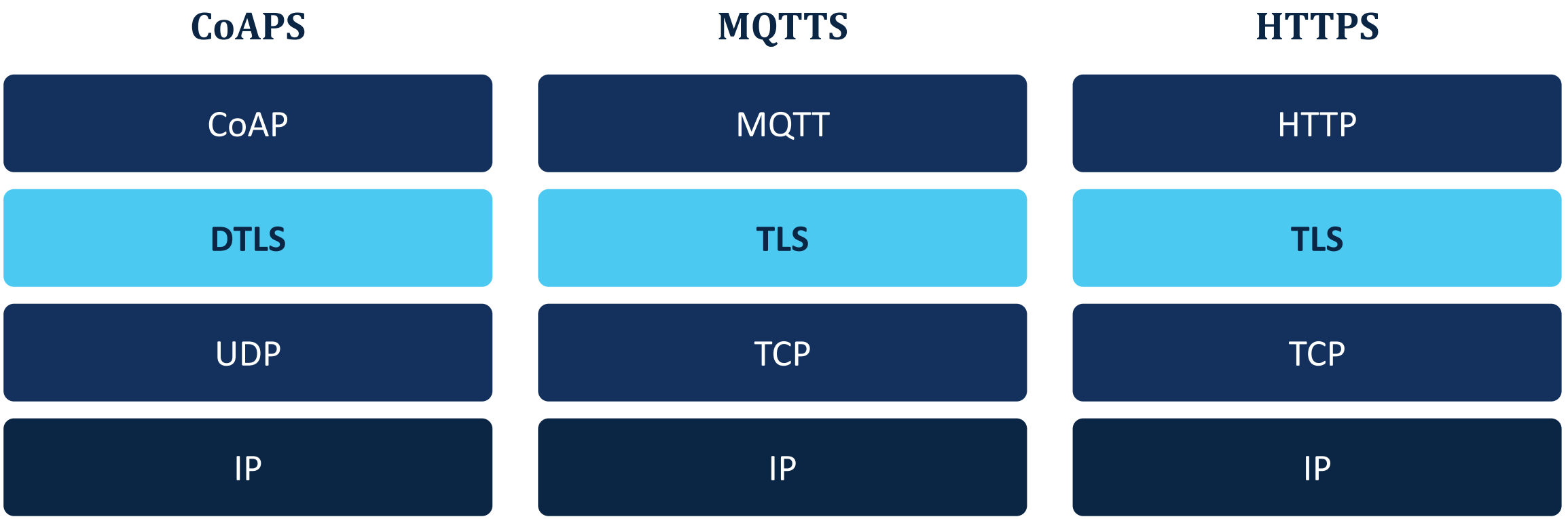
DTLS Record Layer — New Fields for a Datagram World



Highlighted fields are new compared to the TLS record header

- Epoch — increments every time the keys change (e.g., once the handshake completes)
- Sequence number — 48-bit counter; lets the receiver detect reordering and duplicates
- Replay window — receiver keeps a sliding window and drops old or repeated records

TLS / DTLS Across IoT Protocol Stacks



Security protocol choice follows the transport below it — TCP pairs with TLS, UDP pairs with DTLS.

DEPLOYMENT PRACTICE

Lightweight Profiles for Constrained Devices

1	**PSK cipher suites**	Pre-shared keys skip certificate parsing and PKI overhead entirely.
2	**ECC (ECDHE / ECDSA)**	Smaller keys and far less compute than RSA for key exchange and signatures.
3	**Session resumption**	PSK-based resumption — with 0-RTT in TLS/DTLS 1.3 — skips the full handshake on reconnect.
4	**Connection ID (DTLS 1.3)**	Keeps the session alive through NAT rebinding or IP changes, avoiding a re-handshake.

Choosing Between TLS and DTLS

Use TLS when…

- A reliable, ordered TCP connection is available
- The application already speaks HTTP or MQTT
- Gateways bridge IoT data to cloud back-ends

Use DTLS when…

- UDP is required for low latency or multicast
- Devices are radio-constrained (e.g., 802.15.4)
- The application speaks CoAP or raw telemetry

Both TLS 1.3 and DTLS 1.3 share the same cryptographic core — the difference is how each copes with the transport beneath it.